Topic 4:
Solidity

Solidity Tutorial

Solidity is usual high-level language for the EVM

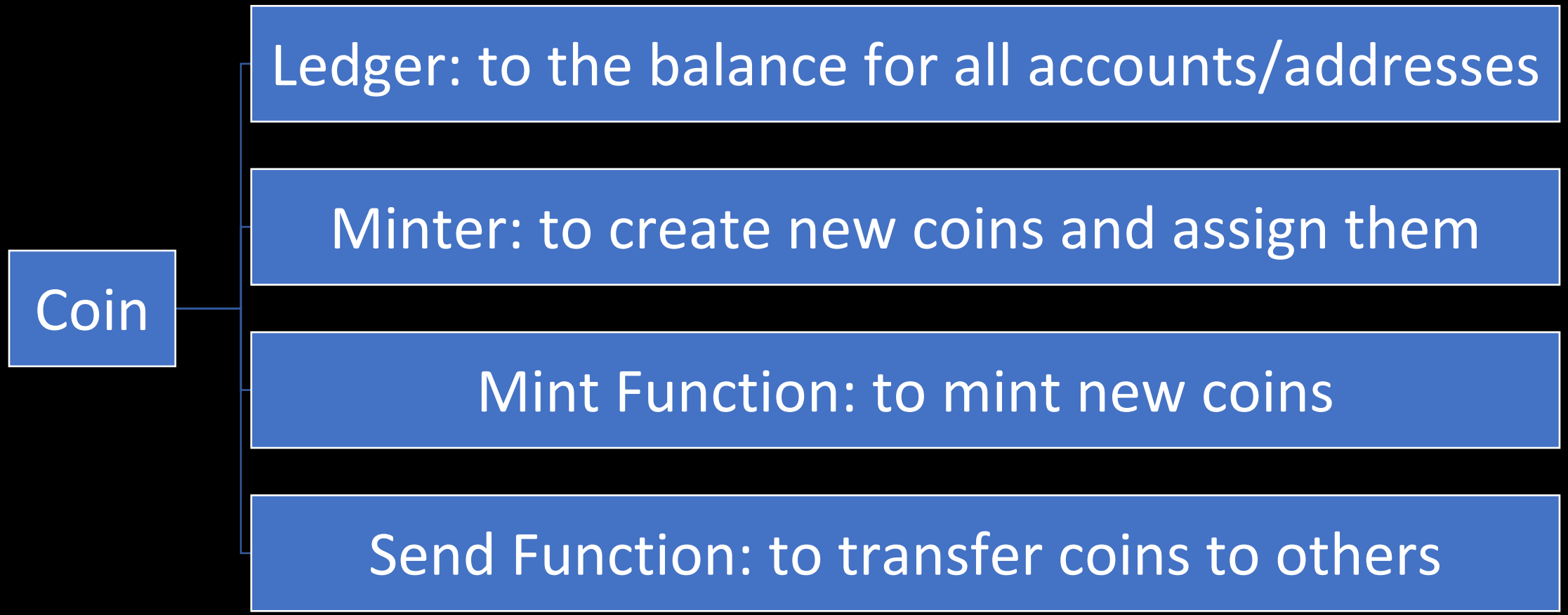
Based on JavaScript

Here, example-based tutorial (may be out-of-date)

Use on-line docs and compiler to learn



Coin Example



Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 …
}
```

Coin Example

This program needs compiler version 0.5.2 or later

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 …
}
```

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
  address minter;
  m                                    s;
  …
}
```

A contract is like a class.
This contract manages a simple coin

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
  address minter;
  mapping (address => uint) balances;
  …
}
```

A contract encompasses long-lived state that lives in the EVM's storage

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
  address minter;
  mapping (address => uint) balances;
  …
}
```

Name of an account (person or contract).

Here, only one allowed to mint new coins.

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 …
}
```

Like a hash table

Initially, every address maps to zero

Tracks client balances

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 constructor() public {
   minter = msg.sender;
 }
}
```

Initializes a contract

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 constructor() public {
   minter = msg.sender;
 }
}
```

address of contract making the call

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 constructor() public {
  minter = msg.sender;
 }
}
```

Remember the creator's address.

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;

 function mint(address owner, uint amount) public {
   if (msg.sender != minter) return;
   balances[owner] += amount;
 }
 …
}
```

A function is like a method.

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (add
 …
 function mint(address owner, uint amount)public {
   if (msg.sender != minter) return;
   balances[owner] += amount;
 }
 …
}
```

Only minter can mint new coin.

If the caller is not authorized, just return.

Otherwise credit the amount to the owner's balance

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 addr
 mapping (address => uint) balances;
 …
 function send(address receiver, uint amount)
  public {
   if (balances[msg.sender] < amount) return;
   balances[msg.sender] -= amount;
   balances[receiver] += amount;
 }
…
}
```

One party transfers coins to counterparty

Coin Example

```
pragma solidity ^0.5.2;
contract Coin {
 address minter;
 mapping (address => uint) balances;
 …
 functio                          r, uint amount)
  public {
   if (balances[msg.sender] < amount) return;
   balances[msg.sender] -= amount;
   balances[receiver] += amount;
 }
…
}
```

Quit if insufficient funds

transfer amount from one account to the other

Coin Flip Game

Alice and Bob want to flip a fair coin

Both of them are willing to cheat

Obvious sources of randomness:
time, block hashes, etc. don't work

Let's try something else

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
    address[2] player;
    mapping (address => uint) vote;
    mapping (address => bool) played;
    bool result;
    bool done;
…
```

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
    address[2] player;
    mapping (address => uint) vote;
    mapping (address => bool) played;
    bool result;
    bool done;

…
```

declare contract and compiler version

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
    address[2] player;
    mapping (address => uint) vote;
    mapping (address => bool) played;
    bool result;
    bool done;
…
```

will want to restrict interactions to the two players

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
    address[2] player;
    mapping (address => uint) vote;
    mapping (address => bool) played;
    bool result;
    bool done;

…
```

each player contributes to the result

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
    address[2] player;
    mapping (address => uint) vote;
    mapping (address => bool) played;
    bool result;
    bool done;

…
```

each player gets one move only

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
    address[2] player;
    mapping (address => uint) vote;
    mapping (address => bool) played;
    bool result;
    bool done;
…
```

remember outcome

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
…
 constructor (address alice, address bob) public {
     player[0] = alice;
     player[1] = bob;
 }
…
```

Coin Flip Example

```
pragma solidity ^0.5.1;
contract CoinFlipMaybe {
…
 constructor (address alice, address bob) public {
     player[0] = alice;
     player[1] = bob;
 }
…
```

constructor just remembers players

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

require(condition)

means abort computation if condition not satisfied.

Like throwing an exception

Exception Check: require() and assert()

```
if(msg.sender != owner) { revert(); }

assert(msg.sender == owner);

require(msg.sender == owner);
```

https://medium.com/blockchannel/the-use-of-revert-assert-and-require-in-solidity-and-the-new-revert-opcode-in-the-evm-1a3a7990e06e

Exception Check: Gas Efficiency

- assert(false) compiles to 0xfe, which is an invalid opcode, using up all remaining gas, and reverting all changes.

- require(false) compiles to 0xfd which is the REVERT opcode, meaning it will refund the remaining gas. The opcode can also return a value (useful for debugging).

Exception Check: Recommended Uses

Use require()

- Validate user inputs
- Validate the response from an external contract
- Validate state conditions prior to executing state changing operations
- used more often, used towards the beginning of a function

Use assert()

- check for overflow/underflow
- check invariants
- validate contract state after making changes
- avoid conditions which should never, ever be possible
- Used less often, used towards the end of your function

Coin Flip Example

```
function play(uint  vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

Only Alice and Bob allowed to play

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

Each player can play only once

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

record player's vote

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] =  vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

record that player voted

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

we are done if both players voted

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

remember we are done

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

take XOR of votes

Coin Flip Example

```
function play(uint _vote) public {
 require(msg.sender == player[0] ||
         msg.sender == player[1]);
 require(!played[msg.sender]);
 vote[msg.sender] = _vote;
 played[msg.sender] = true;
 if (played[player[0]] && played[player[1]]) {
  done = true;
  result = ((vote[player[0]] ^
             vote[player[1]]) % 2) == 1;
 }
}
```

flip value is parity of XOR of votes

Coin Flip Example

```
function getResult() public view returns (bool)
{
    require (done);
    return result;
}
```

Function return variables are declared

promise not to modify the state

Coin Flip Example

```
function getResult() public view returns (bool)
{
    require (done);
    return result;
}
```

anyone can request outcome

Coin Flip Example

```
function getResult() public view returns (bool)
{
    require (done);
    return result;
}
```

make sure outcome is ready

This Contract is Insecure

Suppose Alice plays first

Bob then observes contract state on Etherscan.io

Picks his vote after seeing hers

Bob controls coin flip outcome

Commit-Reveal Pattern

Step One: commit to a value *without revealing it*

you cannot change your value

Step Two: Wait for others to commit

Step Three: reveal your value

you can check validity

Commit-Reveal Pattern

Alice generates a random value r

Alice commits to r by sending hash(r) to contract

Alice waits for Bob to commit

Alice reveals r to contract

Compute outcome when Bob reveals

All Done Off-Chain!

Commit-Reveal Flip

```
contract CoinFlipCR {
    address[2] player;
    mapping (address => uint) commitment;
    mapping (address => uint) revelation;
    mapping (address => bool) committed;
    mapping (address => bool) revealed;
    bool result;
    bool done;
```

Commit-Reveal Flip

```
contract CoinFlipCR {
    address[2] player;
    mapping (address => uint) commitment;
    mapping (address => uint) revelation;
    mapping (address => bool) committed;
    mapping (address => bool) revealed;
    bool result;
    bool done;
```

track both commit and reveal

Commit-Reveal Flip

```
function commit(uint _commit) public {
    require(msg.sender == player[0] ||
        msg.sender == player[1]);
    require(!committed[msg.sender]);
    commitment[msg.sender] = _commit;
    committed[msg.sender] = true;
   }
```

Commit-Reveal Flip

```
function commit(uint _commit) public {
    require(msg.sender == player[0] ||
            msg.sender == player[1]);
    require(!committed[msg.sender]);
    commitment[msg.sender] = _commit;
    committed[msg.sender] = true;
  }
```

members only

Commit-Reveal Flip

```
function commit(uint _commit) public {
    require(msg.sender == player[0] ||
            msg.sender == player[1]);
    require(!committed[msg.sender]);
    commitment[msg.sender] = _commit;
    committed[msg.sender] = true;
  }
```

You only commit once

Commit-Reveal Flip

```
function commit(uint _commit) public {
    require(msg.sender == player[0] ||
        msg.sender == player[1]);
    require(!committed[msg.sender]);
    commitment[msg.sender] = _commit;
    committed[msg.sender] = true;
}
```

close the deal

Coin Flip Example

```
function reveal(uint _reveal) public {
 require(msg.sender == player[0] ||
            msg.sender == player[1]);
 require(!revealed[msg.sender]);
 require(
   commitment[msg.sender]==hash(_reveal));
 revelation[msg.sender] = _reveal;
 revealed[msg.sender] = true;
 if (revealed[player[0]] &&
     revealed[player[1]]) {
  done = true;
  result = ((revelation[player[0]] ^
             revelation[player[1]])
                  % 2) == 1;
 }
}
```

Coin Flip Example

```
function reveal(uint _reveal) public {
  require(msg.sender == player[0] ||
          msg.sender == player[1]);
  require(!revealed[msg.sender]);
  require(
    commitment[msg.sender]==hash(_reveal));
  revelation[msg.sender] = _reveal;
  revealed[msg.sender] = true;
  if (revealed[player[0]] &&
      revealed[player[1]]) {
   done = true;
   result = ((revelation[player[0]] ^
              revelation[player[1]])
                   % 2) == 1;
  }
}
```

Usual preconditions

Coin Flip Example

```
function reveal(uint _reveal) public {
 require(msg.sender == player[0] ||
             msg.sender == player[1]);
 require(!revealed[msg.sender]);
 require(
   commitment[msg.sender]==hash(_reveal));
 revelation[msg.sender] = _reveal;
 revealed[msg.sender] = true;
 if (revealed[player[0]] &&
     revealed[player[1]]) {
  done = true;
  result = ((revelation[player[0]] ^
             revelation[player[1]])
                  % 2) == 1;
 }
}
```

make sure reveal is sincere

Coin Flip Example

```
function reveal(uint _reveal) public {
 require(msg.sender == player[0] ||
            msg.sender == player[1]);
 require(!revealed[msg.sender]);
 require(
   commitment[msg.sender]==hash( reveal));
 revelation[msg.sender] = _reveal;
 revealed[msg.sender] = true;
 if (revealed[player[0]] &&
     revealed[player[1]]) {
  done = true;
  result = ((revelation[player[0]] ^
             revelation[player[1]])
                  % 2) == 1;
 }
}
```

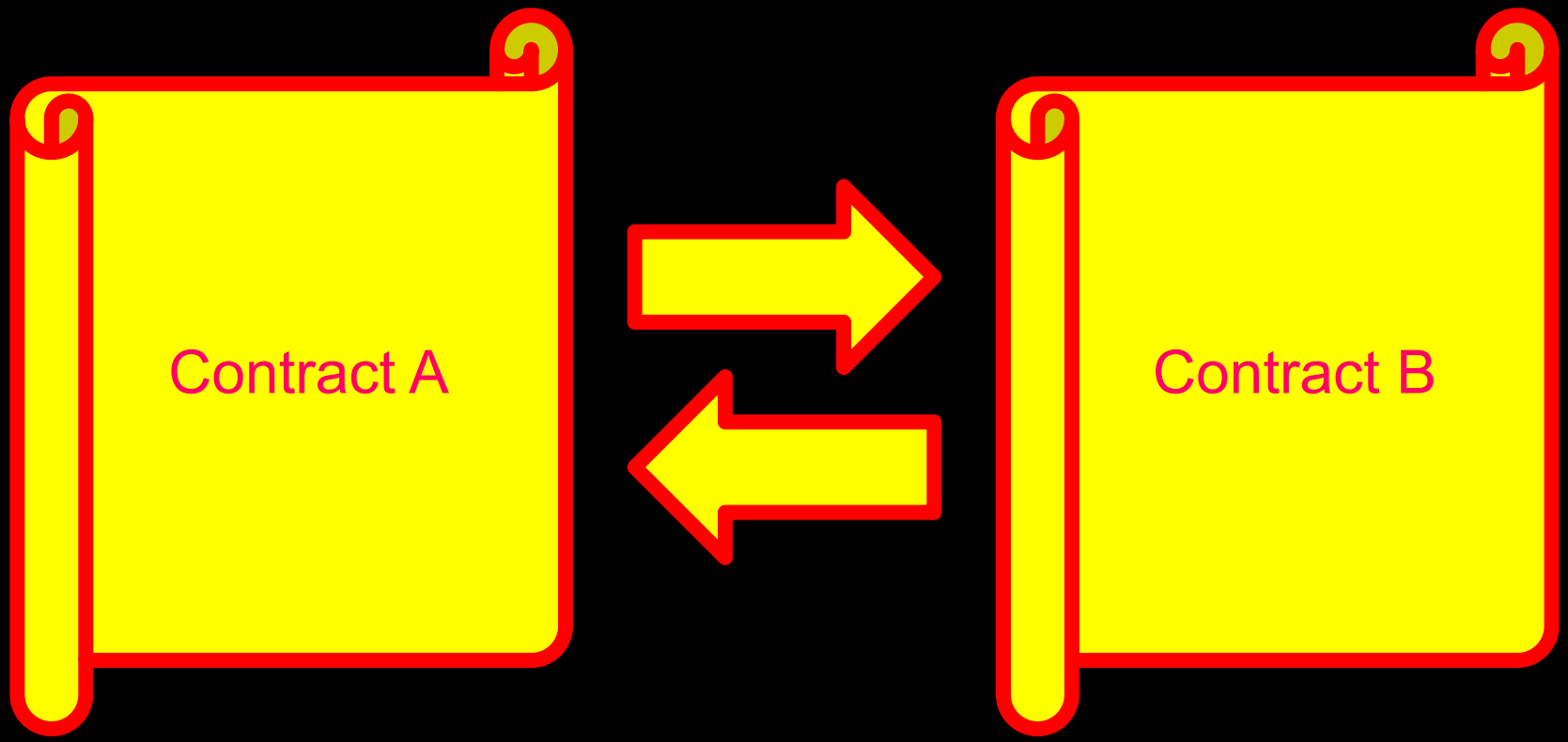
record revelation

Coin Flip Example

```
function reveal(uint _reveal) public {
 require(msg.sender == player[0] ||
            msg.sender == player[1]);
 require(!revealed[msg.sender]);
 require(
   commitment[msg.sender]==hash(_reveal));
 revelation[msg.sender] = _reveal;
 revealed[msg.sender] = true;
 if (revealed[player[0]] &&
     revealed[player[1]]) {
  done = true;
  result = ((revelation[player[0]] ^
             revelation[player[1]])
                  % 2) == 1;
 }
}
```

If done, compute outcome as before

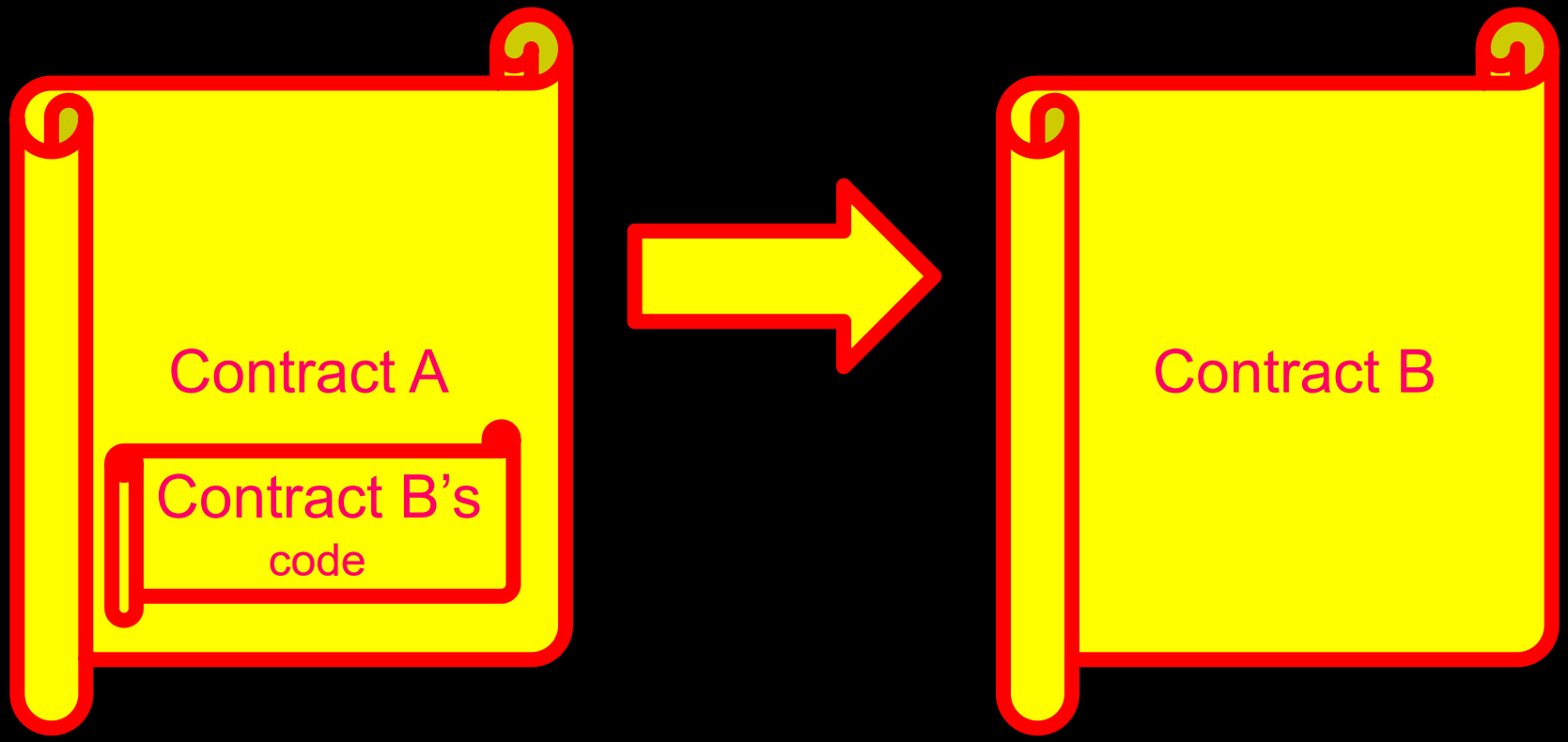
Call Opcode



A calls B’s function …

which runs in B’s address space

DelegateCall Opcode



A delegatecalls B's function …

which runs in A's address space

Library

DRY (Don’t Repeat Yourself)

- Static function in static class
- Syntactic sugar

Contract where every function call is a delegate call

Library

promise not to read from or modify the state

```
function operate(uint a, uint b) pure
   returns(uint,uint,uint,uint){
      uint add = a+b;
      uint sub = a-b;
      uint mul = a*b;
      uint div = a/b;
}
```

Error when b = 0

Library

```
library SafeMath{

  function mul(uint a,uint b) pure
     returns(uint){
        uint c = a*b;
        assert(c/a==b);
        return c;
  }

  function div(uint a,uint b) pure
     returns(uint){
        uint c = a/b;
        assert(a== b*c +a%b);
        return c;
  }
}
```

```
  function sub(uint a,uint b) pure
     returns(uint){
        assert(a>=b);
        return a-b;
  }

  function add(uint a,uint b) pure
     returns(uint){
        uint c = a+b;
        assert(c>=a);
        return c;
  }
```

Library

```
function operate(uint a, uint b)
  pure returns (uint,uint,uint,uint){
    uint plus = SafeMath.add(a,b);
    uint minus = SafeMath.sub(a,b);
    uint multiply =SafeMath.mul(a,b);
    uint divide = SafeMath.div(a,b);
    return(plus, minus, multiply, divide);
}
```

Library

```
//make SafeMath a library for unsigned int
using SafeMath for uint;
function operate2(uint a, uint b) pure
  returns(uint,uint,uint,uint){
    uint plus = a.add(b);
    uint minus = a.sub(b);
    uint multiply =a.mul(b);
    uint divide = a.div(b);
    return(plus, minus, multiply, divide);
}
```

Abstract Contracts

```
contract Config
{
 // defined
 function name() public returns (string) return “Config”;
 // undefined
 function lookup(uint id) public returns (address);
}
```

At least one function implementation missing

Compiler type-checking, inheritance

Interfaces

```
interface Config {
  function lookup(uint id) public returns (address);
}
```

All function implementations missing

Inheritance

```
contract Feline {
 function utterance()
   public returns (bytes32);
 }

contract Cat is Feline {
 function utterance()
   public returns (bytes32) {
     return "meow";
 }
}
```

Like other languages

Function Overloading

```
pragma solidity >=0.4.16 <0.6.0;
contract A {

 function f(uint _in) public pure
   returns (uint out) {
     out = _in;
 }
 function f(uint _in, bool _really) public pure
   returns (uint out) {
     if (_really) out = _in;
 }

}
```

Function Visibility

External: only from outside

Public: only internal or external

Internal: only current or derived contracts

Private: only current not derived contracts

Getter Functions

```
pragma solidity >=0.4.0 <0.6.0;

contract C {
  uint public data = 42;
}

contract Caller {
   C c = new C();
   function f() public view returns (uint) {
      return c.data();
   }
}
```

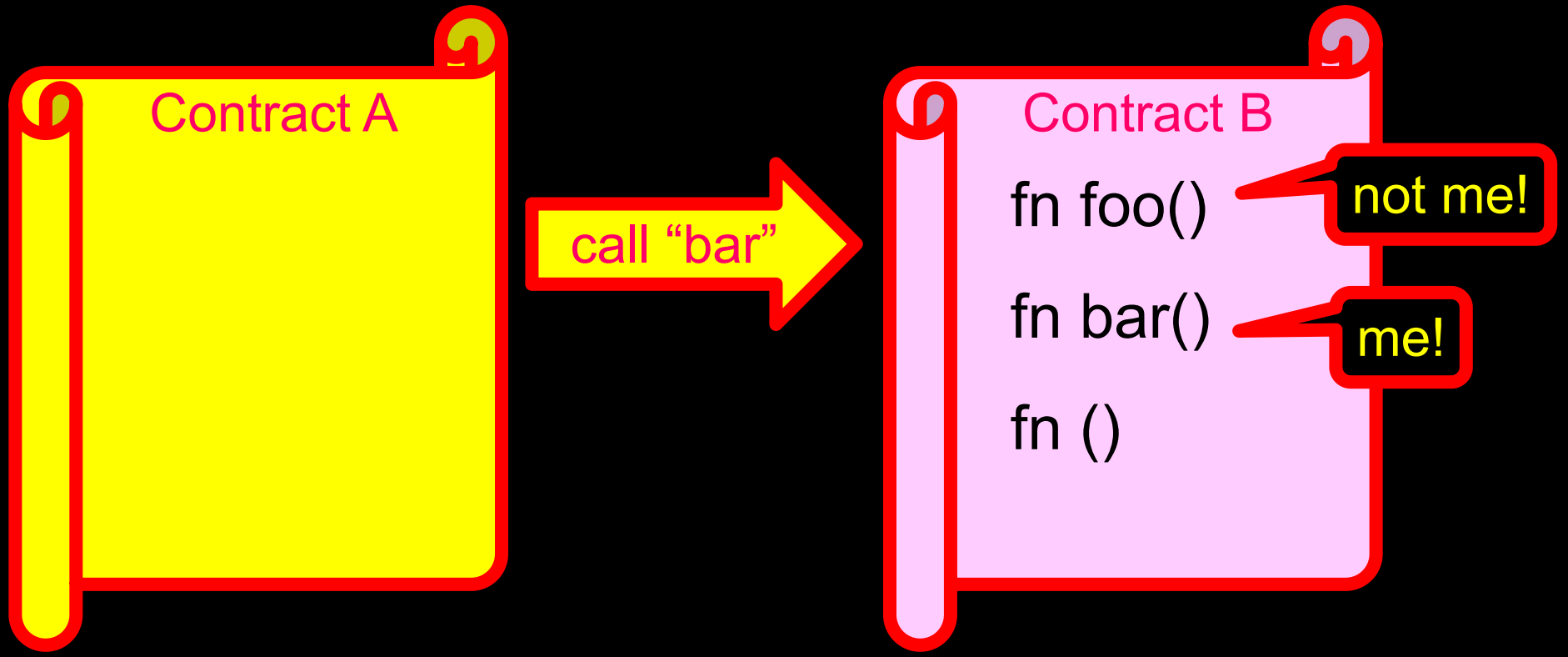
Automatically created by compiler for all public state variables.

Fallback Function: Motivation

Transaction data in a contract call includes:

- Function selector (hashed name)
- Function arguments

Function Call



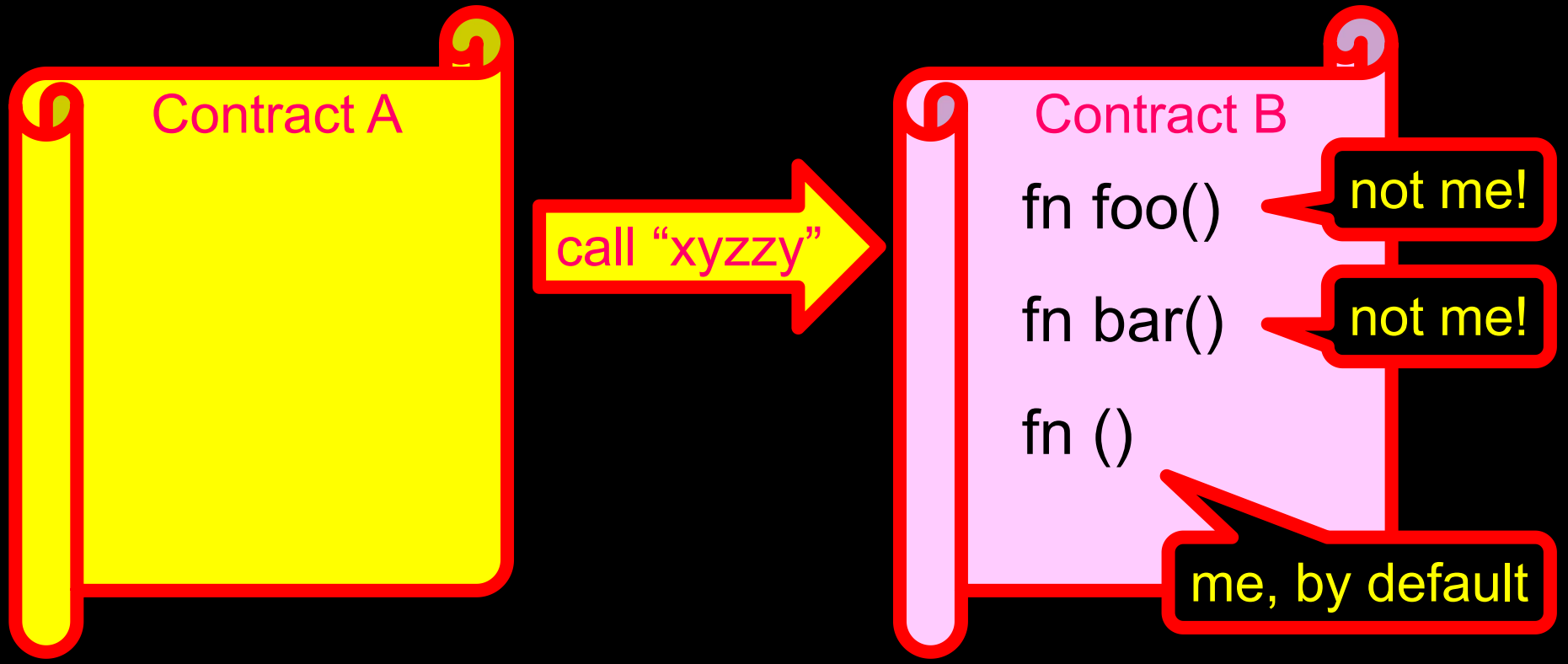
A calls B’s “bar” function …

Fallback Function: Motivation

What if …

- The transaction matched no functions or no data supplied
- whenever the contract receives plain Ether (without data)

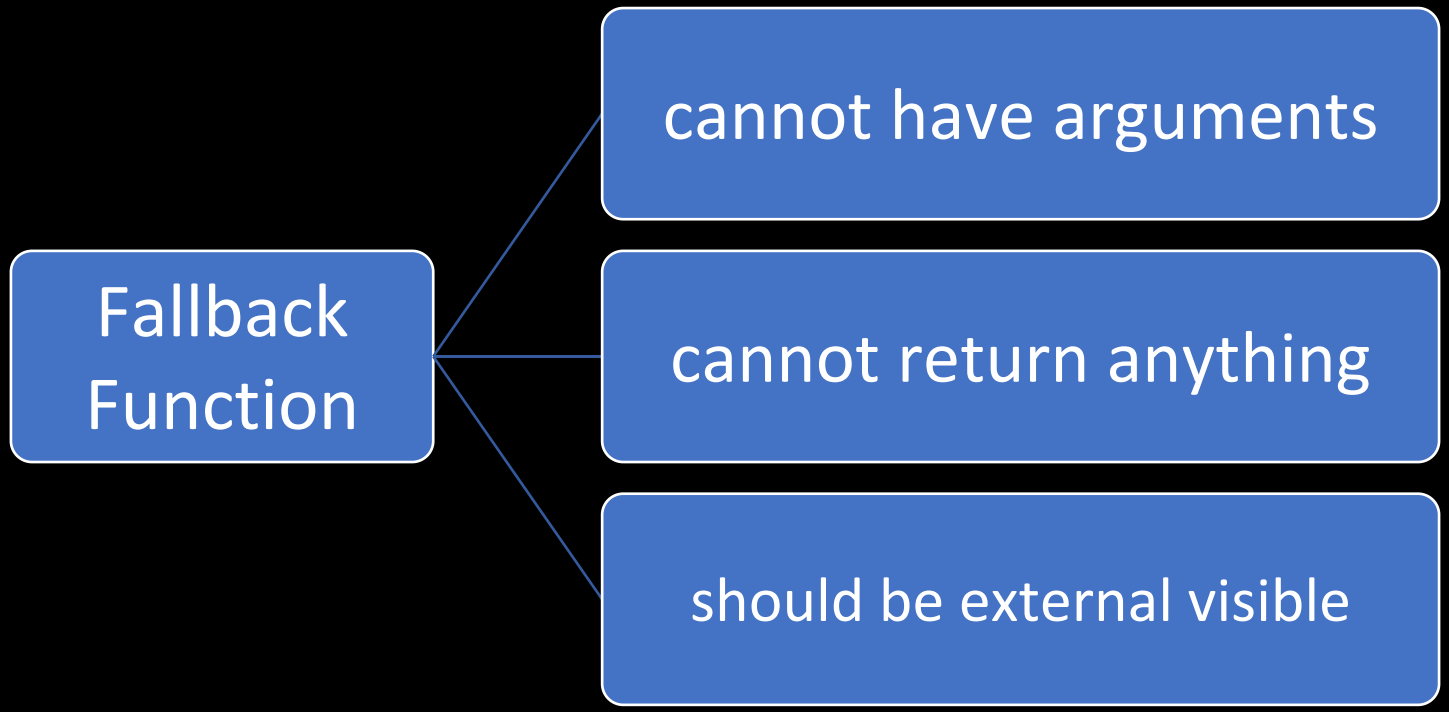
Fallback Function



A calls B’s “xyzzy” function …

Fallback Function

A contract can have exactly one unnamed function



Fallback Function

```
contract Example {
  uint x = 0;
  function foo() public {
    x = 1;
  }
  function () public payable {
    x = 2;
  }
}
```

named regular function

Fallback Function

```
contract Example {
  uint x = 0;
  function foo() public {
    x = 1;
  }
  function () public payable {
    x = 2;
  }
}
```

named regular function

nameless fallback function

Often used to receive payments

Fallback Function

```
contract Example {
  uint x = 0;
  function foo() public {
    x = 1;
  }
  function () public payable {
    x = 2;
  }
}
```

allows a function to receive ether while being called

If you try to send ether to a function without a payable modifier, the transaction will be rejected.

Three Methods to Send Ether

```
address.send(amount)
```

```
address.transfer(amount)
```

```
address.call.value(amount)( )
```

The DAO Attacked: Code Issue Leads to $60 Million Ether Theft

This happened

Michael del Castillo (@DelRayMan) | Published on June 17, 2016 at 14:0

380 742 16 330 1



The DAO, the distributed autonomous organization that had collected over $150m worth of the cryptocurrency ether, has reportedly been hacked, sparking a broad market sell-off.

A leaderless organization comprised of a series of smart contracts written on the ethereum codebase, The DAO has lost 3.6m ether, which is currently sitting in a separate wallet a
separate grouping dubbed

Ether markets plunged on th
Poloniex. With ether current
cryptocurrency at more thar

News of the hack first begar
prompting Ethereum co-founder Vitalik Buterin to call for a pause in trading in ether markets as well as those for DAO tokens, which are used to vote on funding proposals for the decentralized group.

TheDAO was launched in May using open-source code written by Slock.it, an Ethereum-focused startup based in Germany. The DAO is designed to operate like a venture capital fund empowering it members to fund ethereum projects.

To become a voting member of The DAO tokens were sold in exchange for ether during a month-long "creation phase" that resulted in $162m worth of ether being raised. A measure was built into

" The attack is a *recursive calling vulnerability*, where an attacker called the "split" function, and then calls the split function recursively …"

BUSINESS INSIDER TECH

Digital currency Ethereum is cratering because of a $50 million hack



Rob Price

Jun. 17, 2016, 5:34 AM 45,261 11

FACEBOOK LINKEDIN TWITTER

The value of the digital currency Ethereum has dropped dramatically



Martin Hunter/Getty Images

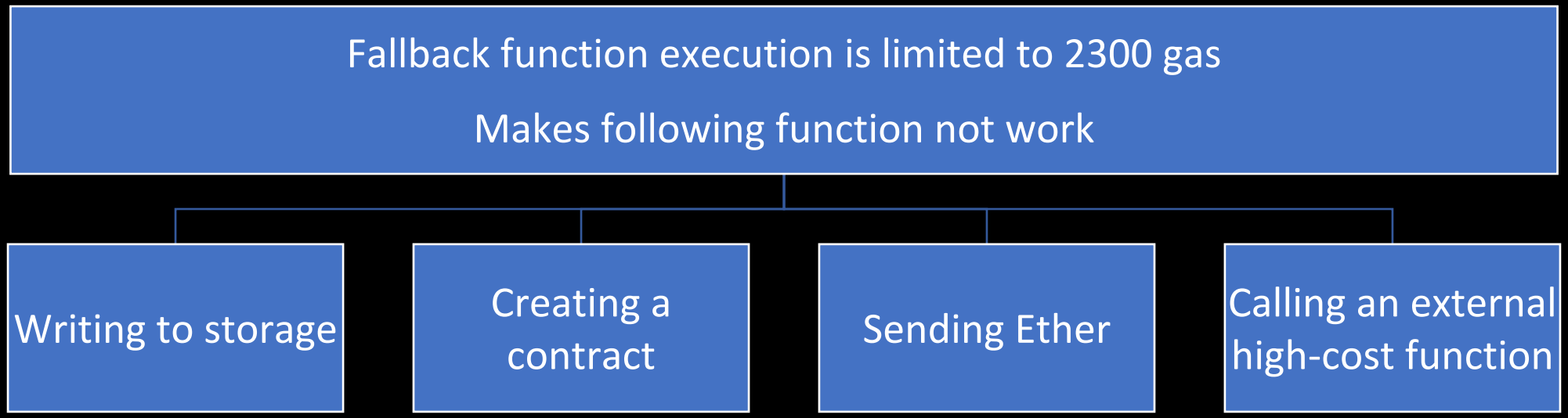
dropped to $15 from record highs of $21.50 in

The actual fix?

as much as $50 million stolen at post-

Fallback Function Idiosyncrasies

```
function () {
    balances[msg.sender] += msg.value;
}
```



Fallback Function Idiosyncrasies

```
function () {
 revert();
}
function deposit() external {
  balances[msg.sender] += msg.value;
}
```

this works because no 2300 gas limit on deposit

Sending Ether to a Payable Fallback Function

```
address.send(amount);
```

Limited to 2300 gas

- Protects against re-entrancy attack
- requires near-trivial fallback function

If transfer fails

- returns Boolean

  (not exception)
- Programming pitfall

Sending Ether to a Payable Fallback Function

```
address.transfer(amount);
```

Limited to 2300 gas

- Protects against re-entrancy attack
- requires near-trivial fallback function

If transfer fails

- reverts transaction
- throws exception
- Safer programming practice

Sending Ether to a Payable Fallback Function

```
address.call.value(amount).gas(extra)();
```

Can specify how much gas

No re-entrancy protection

non-trivial fallback functions OK

Function Modifiers

```
modifier ownerOnly() {
    require (msg.sender == owner);
    _;
}
```

Execute function body

If test passes …

Function Modifiers

If test passes …

```
function pay(address _payto, uint amount) public
  ownerOnly() {
    balance[_payTo] += amount;
}
```

Execute function body

Self Destruction

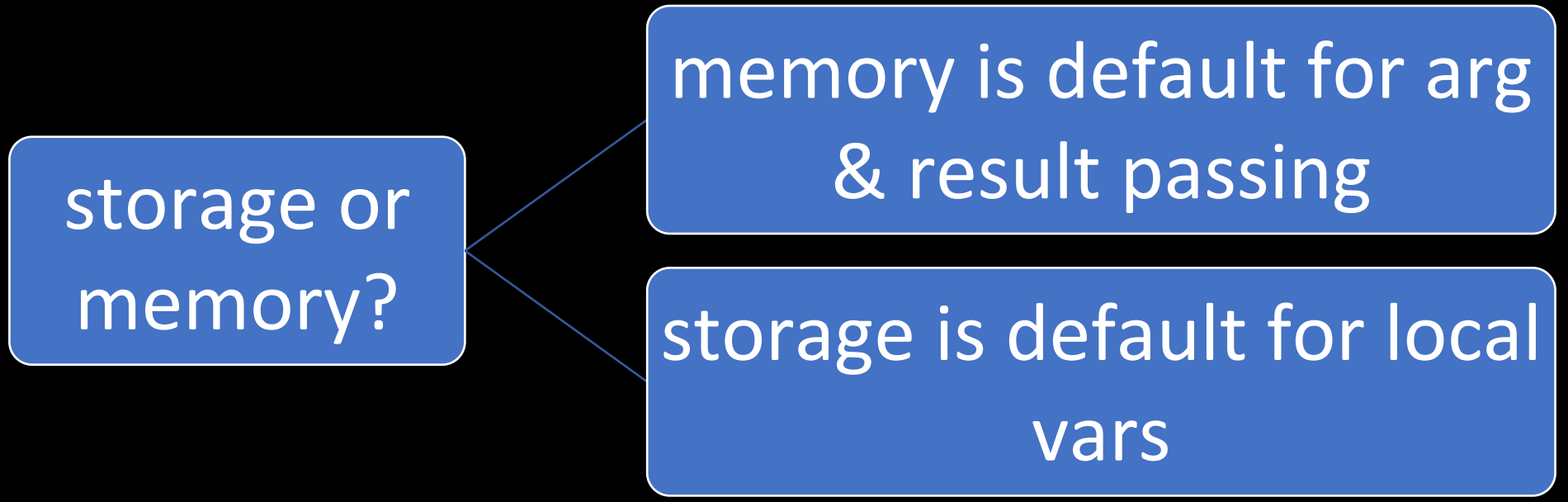
`selfDestruct(sendMoneyTo)`

terminates contract

sends remaining balance to argument

beneficiary cannot refuse

Data Location



assignment across regions is copying

https://programtheblockchain.com/posts/2018/03/09/understanding-ethereum-smart-contract-storage/

Events (a.k.a. Logs)

A public record that something happened

An abstraction on top of the EVM’s logging functionality

Events can be monitored from outside the blockchain

Applications can subscribe and listen to events through API

Logging Operation in EVM

a0s: Logging Operations

For all logging operations, the state change is to append an additional log entry on to the substate's log series:

$$\text{A } 1A'_{\mathbf{l}} \equiv A_{\mathbf{l}} \cdot (I_{\mathrm{a}}, \mathbf{t}, \boldsymbol{\mu}_{\mathbf{m}}[\boldsymbol{\mu}_{\mathbf{s}}[0] \ldots (\boldsymbol{\mu}_{\mathbf{s}}[0] + \boldsymbol{\mu}_{\mathbf{s}}[1] - 1)])$$

and to update the memory consumption counter:

$\boldsymbol{\mu}'_{\mathrm{i}} \equiv M(\boldsymbol{\mu}_{\mathrm{i}}, \boldsymbol{\mu}_{\mathbf{s}}[0], \boldsymbol{\mu}_{\mathbf{s}}[1])$

The entry's topic series, $\mathbf{t}$, differs accordingly:

Value	Mnemonic	δ	α	Description
0xa0	LOG0	2	0	Append log record with no topics. $\mathbf{t} \equiv ()$
0xa1	LOG1	3	0	Append log record with one topic. $\mathbf{t} \equiv (\boldsymbol{\mu}_{\mathbf{s}}[2])$
⋮	⋮	⋮	⋮	⋮
0xa4	LOG4	6	0	Append log record with four topics. $\mathbf{t} \equiv (\boldsymbol{\mu}_{\mathbf{s}}[2], \boldsymbol{\mu}_{\mathbf{s}}[3], \boldsymbol{\mu}_{\mathbf{s}}[4], \boldsymbol{\mu}_{\mathbf{s}}[5])$

Events

Define an Event:

```
event Deposit(address from, uint value);
```

Call or Fire the Event:

```
emit Deposit(msg.sender, msg.value);
```

Example of Event

```
pragama solidity 0.5.3;
contract ExampleContract {

    event Transfer(address indexed _from, address indexed _to, uint256 value);

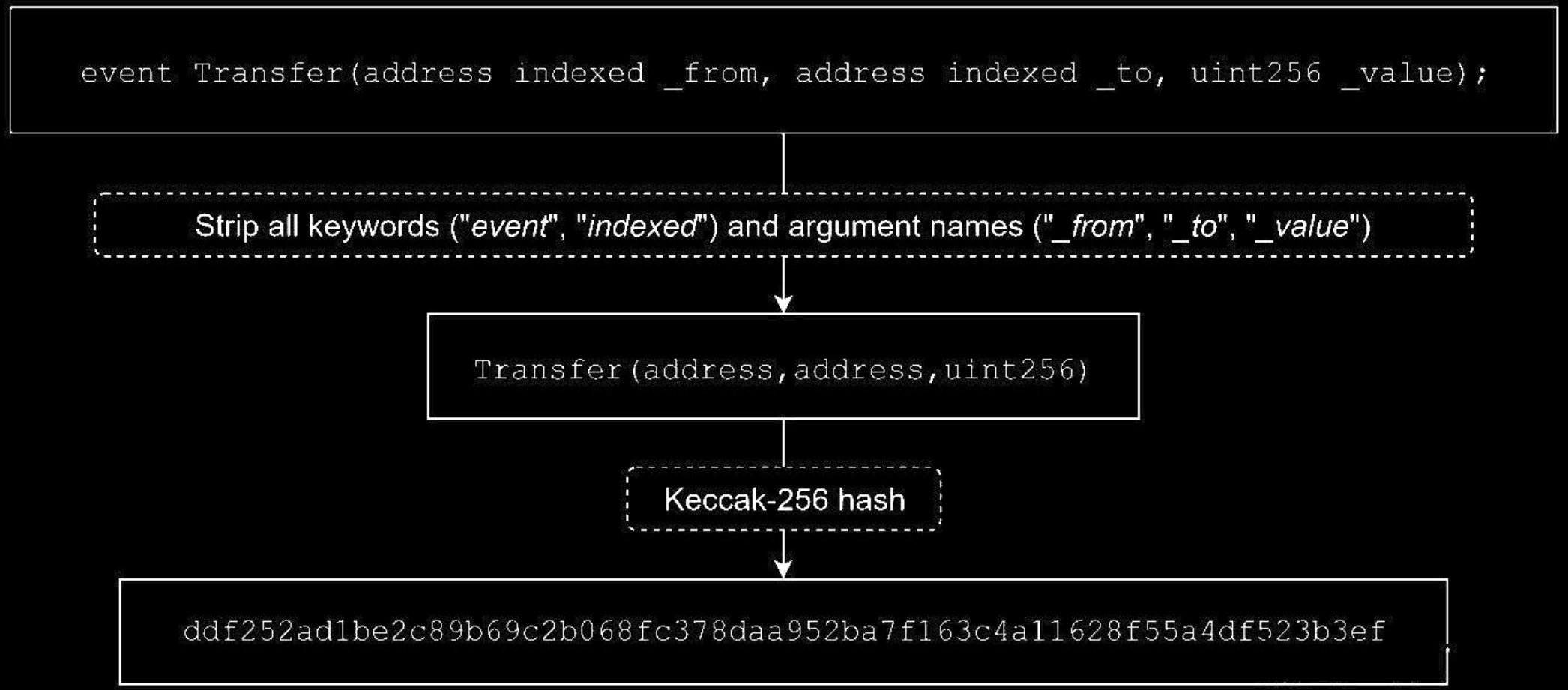
    function transfer(address _to, uit256 _value) public {

        emit Transfer(msg.sender, _to, _value);

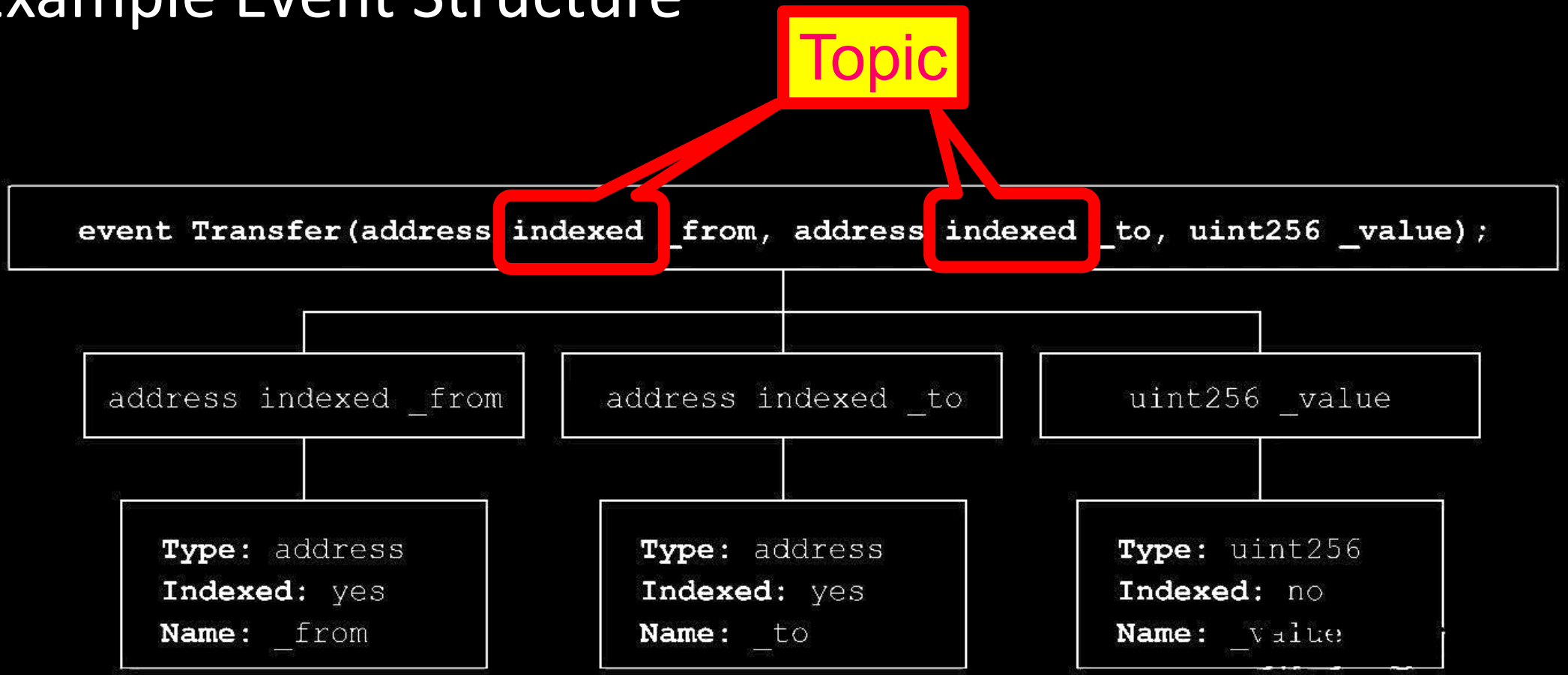
    }

}
```

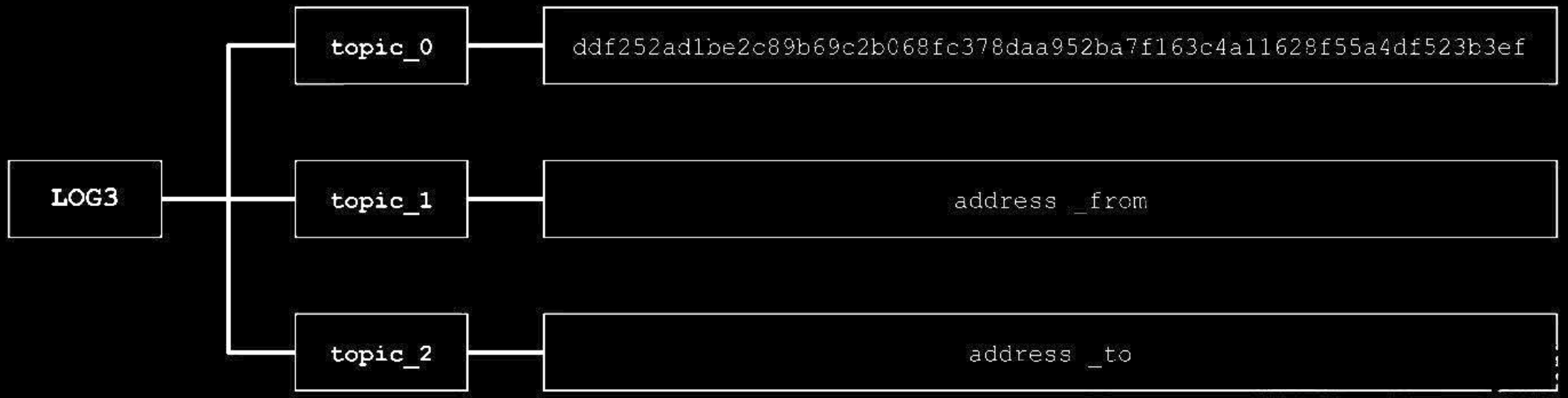
Example Event Signature



Example Event Structure



Example Event in EVM



Front End with Web3.js

```
const Web3 = require("web3");
const web3 = new Web3("wss://mainet.infura.io/ws");


web3.eth.subscribe("logs", {
      address: "0x89d24A6b4CcB1B6fAA2625ff562bDD9a23260359",
      topics: [
            web3.utils.sha3("Transfer(address,address,uint256)")
      ]
}, (error, result) => {
      if(error) console.error(error);
      else console.log(result);
});
```

Gas Cost for Logging

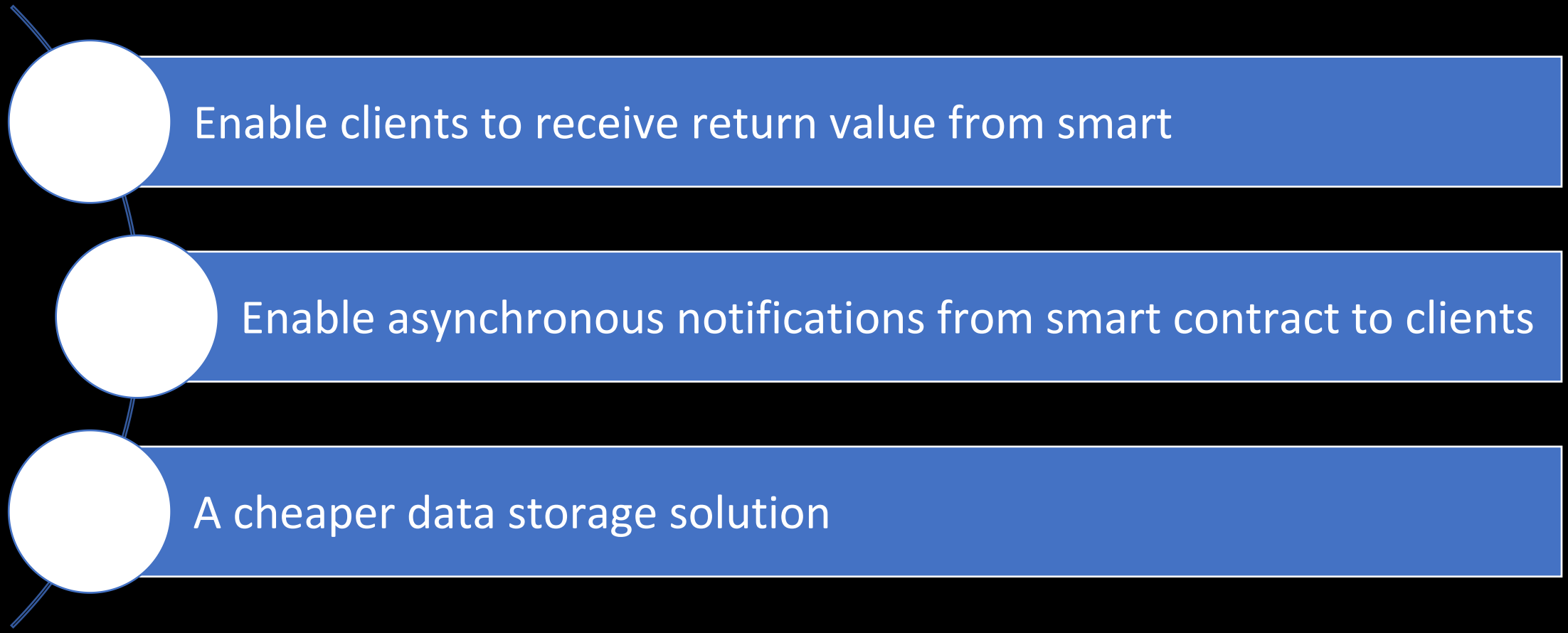
G_{log}	375	Partial payment for a LOG operation.
G_{logdata}	8	Paid for each byte in a LOG operation's data.
G_{logtopic}	375	Paid for each topic of a LOG operation.

$$
\begin{cases}
G_{log} + G_{logdata} \times \boldsymbol{\mu}_{\mathbf{s}}[1] & \text{if } w = \text{LOG0} \\
G_{log} + G_{logdata} \times \boldsymbol{\mu}_{\mathbf{s}}[1] + G_{logtopic} & \text{if } w = \text{LOG1} \\
G_{log} + G_{logdata} \times \boldsymbol{\mu}_{\mathbf{s}}[1] + 2G_{logtopic} & \text{if } w = \text{LOG2} \\
G_{log} + G_{logdata} \times \boldsymbol{\mu}_{\mathbf{s}}[1] + 3G_{logtopic} & \text{if } w = \text{LOG3} \\
G_{log} + G_{logdata} \times \boldsymbol{\mu}_{\mathbf{s}}[1] + 4G_{logtopic} & \text{if } w = \text{LOG4}
\end{cases}
$$

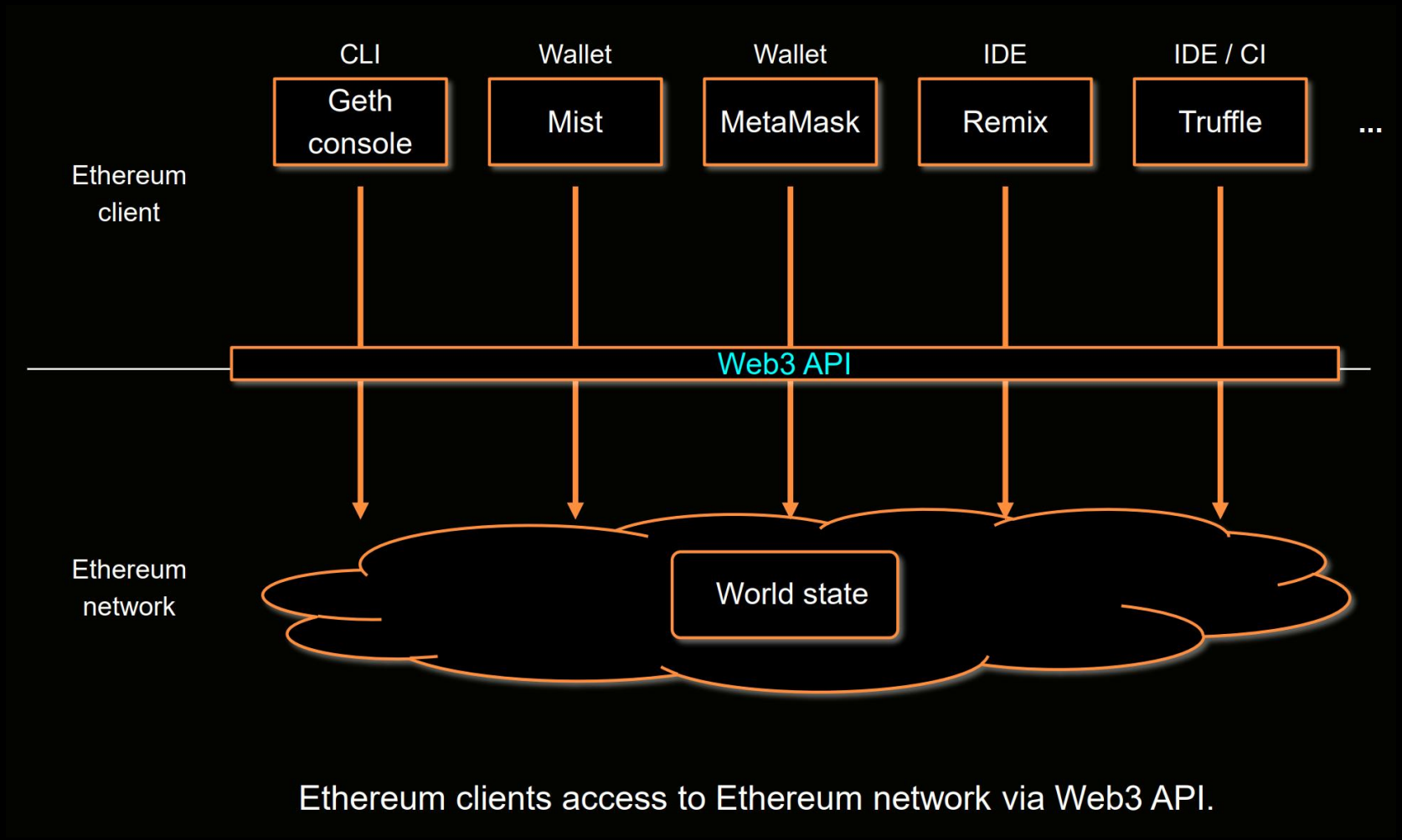
知乎 @Tiny熊

A cheaper way of writing data than transactions (21000+ gas) !

Functionalities of Events

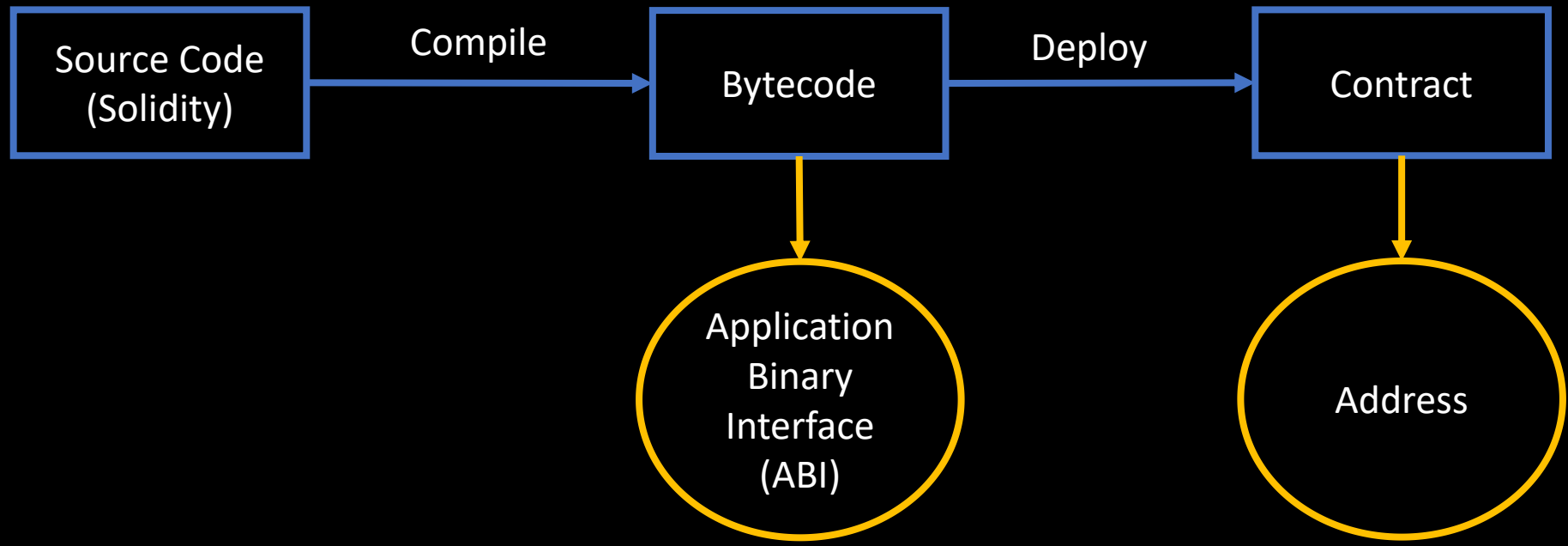


Web3 API

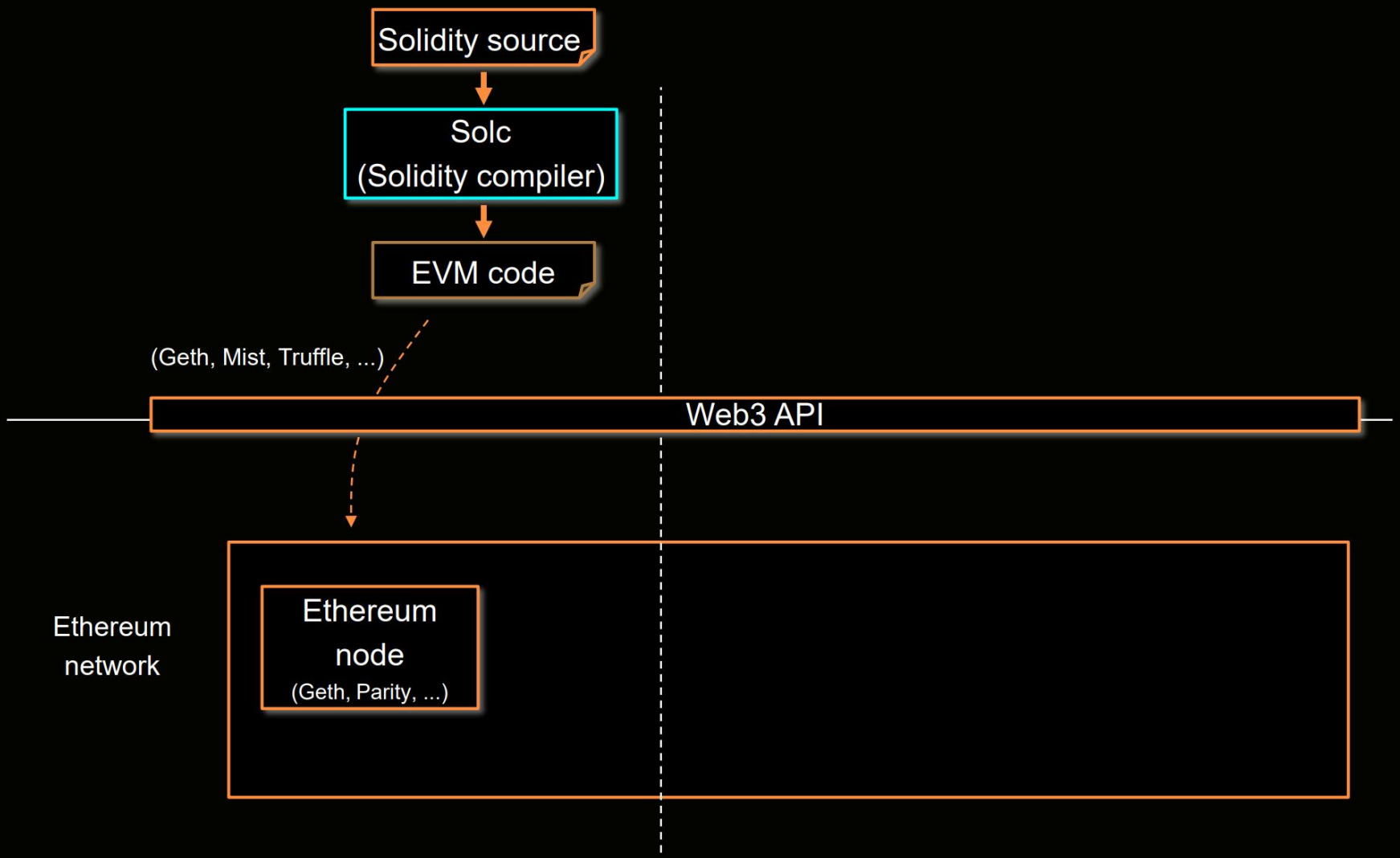


Ethereum clients access to Ethereum network via Web3 API.

Compile and Deploy



Solc


Solidity source
Solc
(Solidity compiler)
EVM code
(Geth, Mist, Truffle, ...)
Web3 API
Ethereum
network
Ethereum
node
(Geth, Parity, ...)

Contract Application Binary Interface (ABI)

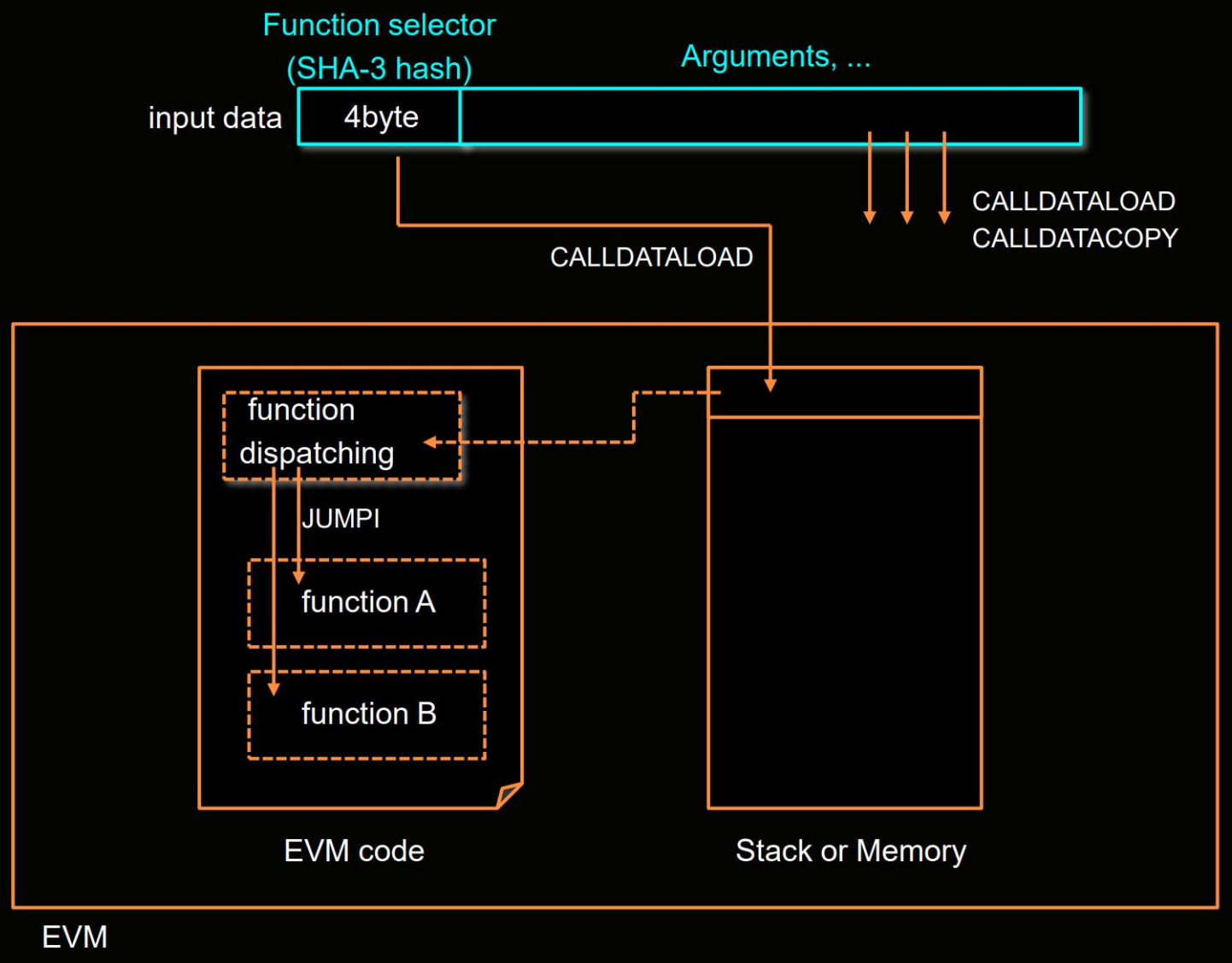
defines

- which functions you can invoke
- what data format will be returned

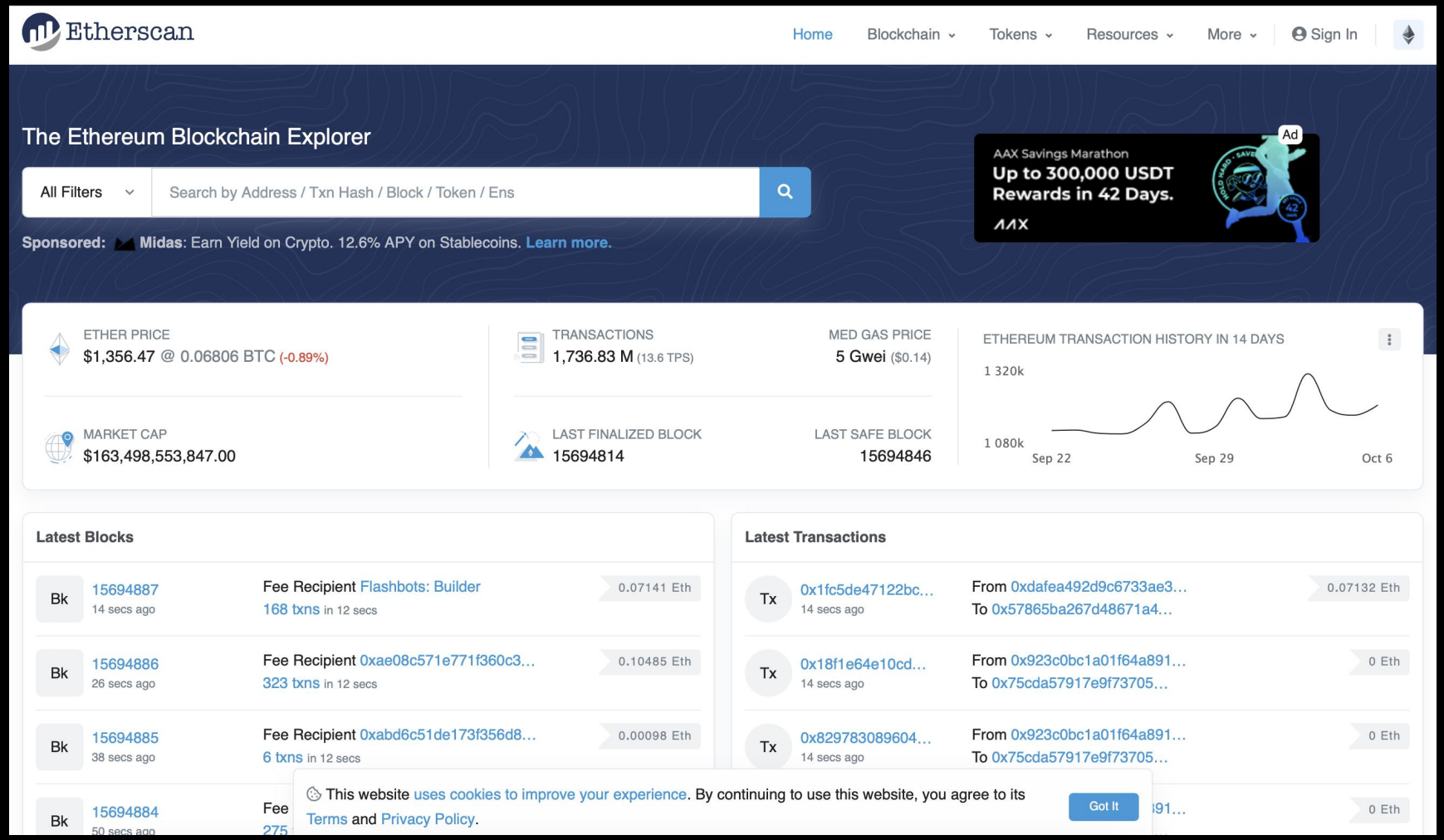
interface for

- contracts invocation from outside the blockchain
- contract-to-contract interaction

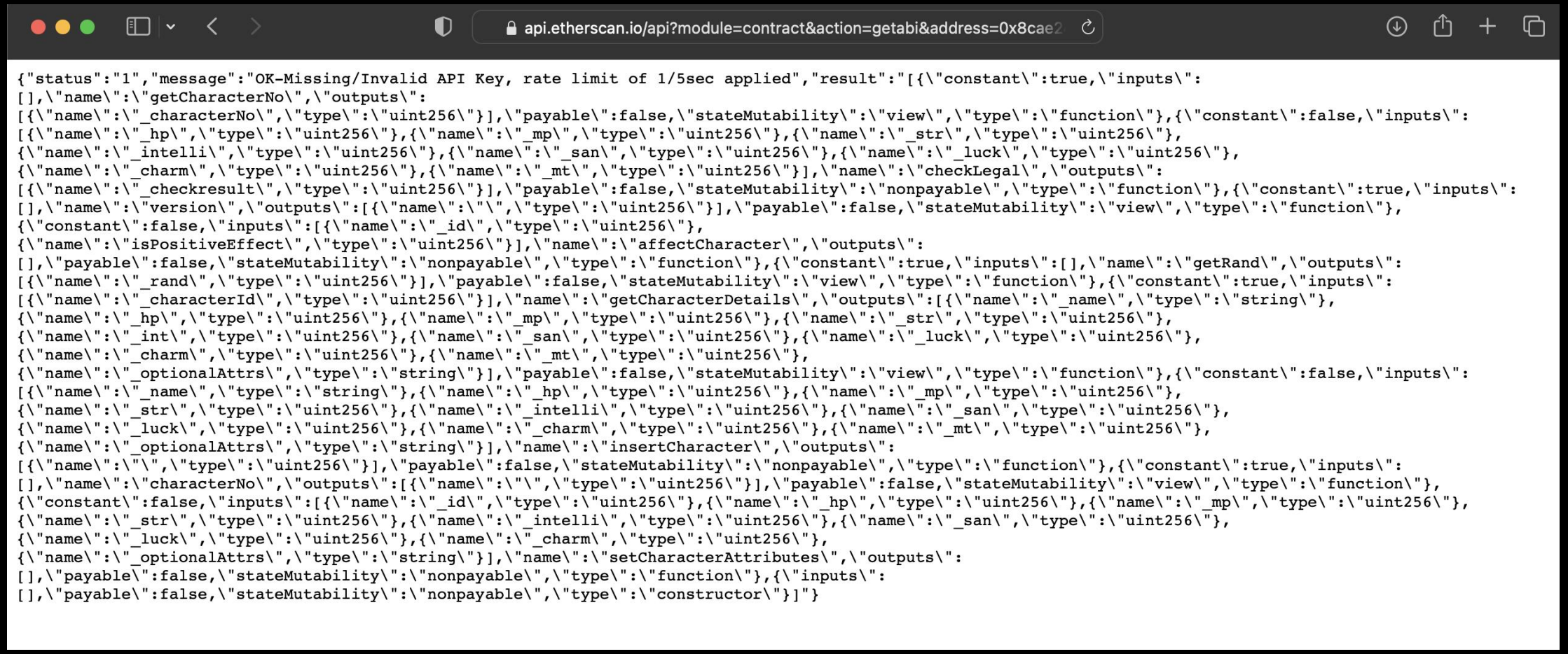
ABI


Function selector
(SHA-3 hash)
Arguments, ...
input data
4byte
CALLDATALOAD
CALLDATACOPY
CALLDATALOAD
function
dispatching
JUMPI
function A
function B
EVM code
Stack or Memory
EVM

Etherscan



https://etherscan.io

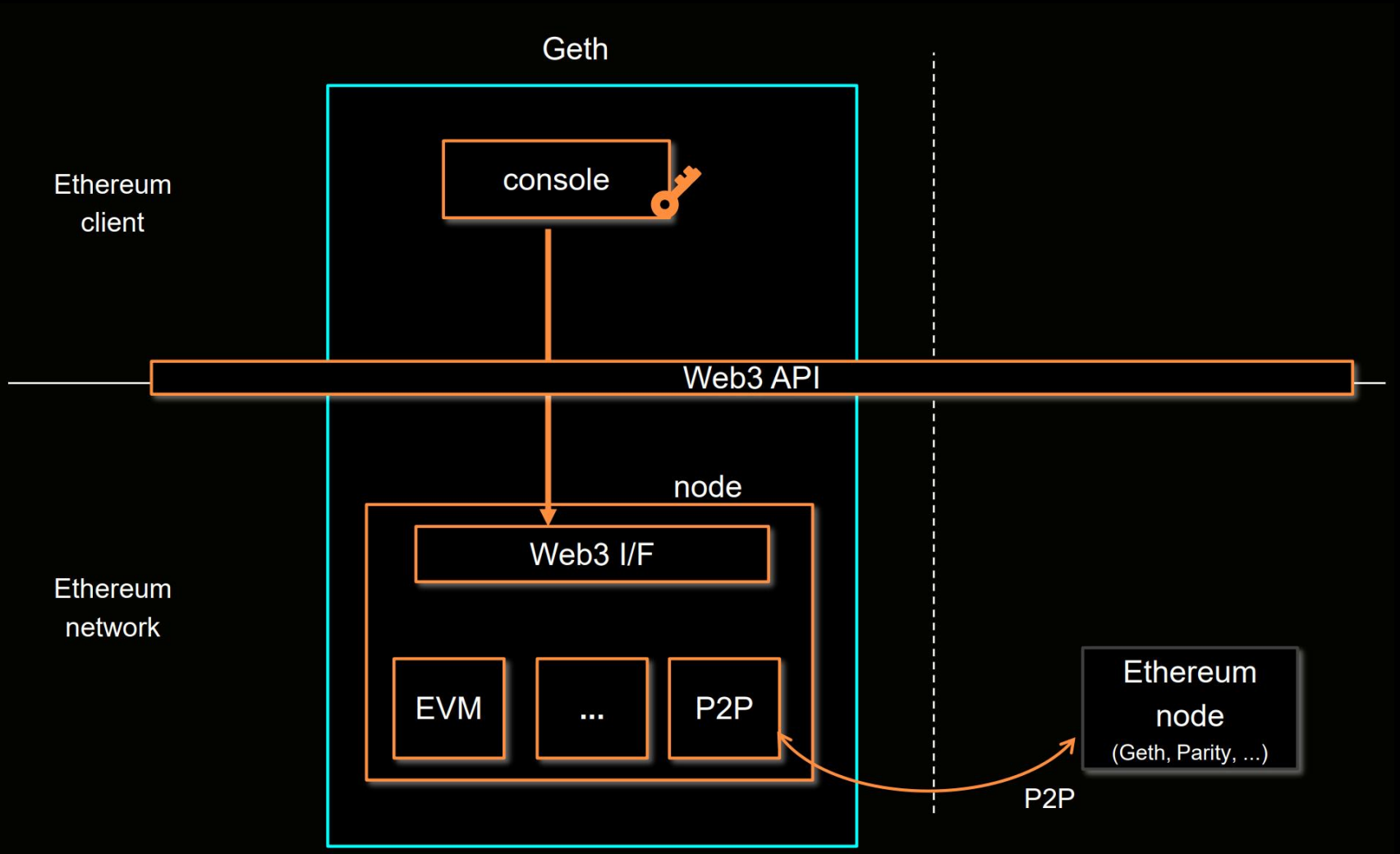



```
{"status":"1","message":"OK-Missing/Invalid API Key, rate limit of 1/5sec applied","result":"[{\"constant\":true,\"inputs\":
[],\"name\":\"getCharacterNo\",\"outputs\":
[{\"name\":\"_characterNo\",\"type\":\"uint256\"}],\"payable\":false,\"stateMutability\":\"view\",\"type\":\"function\"},{\"constant\":false,\"inputs\":
[{\"name\":\"_hp\",\"type\":\"uint256\"},{\"name\":\"_mp\",\"type\":\"uint256\"},{\"name\":\"_str\",\"type\":\"uint256\"},
{\"name\":\"_intelli\",\"type\":\"uint256\"},{\"name\":\"_san\",\"type\":\"uint256\"},{\"name\":\"_luck\",\"type\":\"uint256\"},
{\"name\":\"_charm\",\"type\":\"uint256\"},{\"name\":\"_mt\",\"type\":\"uint256\"}],\"name\":\"checkLegal\",\"outputs\":
[{\"name\":\"_checkresult\",\"type\":\"uint256\"}],\"payable\":false,\"stateMutability\":\"nonpayable\",\"type\":\"function\"},{\"constant\":true,\"inputs\":
[],\"name\":\"version\",\"outputs\":[{\"name\":\"\",\"type\":\"uint256\"}],\"payable\":false,\"stateMutability\":\"view\",\"type\":\"function\"},
{\"constant\":false,\"inputs\":[{\"name\":\"_id\",\"type\":\"uint256\"},
{\"name\":\"isPositiveEffect\",\"type\":\"uint256\"}],\"name\":\"affectCharacter\",\"outputs\":
[],\"payable\":false,\"stateMutability\":\"nonpayable\",\"type\":\"function\"},{\"constant\":true,\"inputs\":[],\"name\":\"getRand\",\"outputs\":
[{\"name\":\"_rand\",\"type\":\"uint256\"}],\"payable\":false,\"stateMutability\":\"view\",\"type\":\"function\"},{\"constant\":true,\"inputs\":
[{\"name\":\"_characterId\",\"type\":\"uint256\"}],\"name\":\"getCharacterDetails\",\"outputs\":[{\"name\":\"_name\",\"type\":\"string\"},
{\"name\":\"_hp\",\"type\":\"uint256\"},{\"name\":\"_mp\",\"type\":\"uint256\"},{\"name\":\"_str\",\"type\":\"uint256\"},
{\"name\":\"_int\",\"type\":\"uint256\"},{\"name\":\"_san\",\"type\":\"uint256\"},{\"name\":\"_luck\",\"type\":\"uint256\"},
{\"name\":\"_charm\",\"type\":\"uint256\"},{\"name\":\"_mt\",\"type\":\"uint256\"},
{\"name\":\"_optionalAttrs\",\"type\":\"string\"}],\"payable\":false,\"stateMutability\":\"view\",\"type\":\"function\"},{\"constant\":false,\"inputs\":
[{\"name\":\"_name\",\"type\":\"string\"},{\"name\":\"_hp\",\"type\":\"uint256\"},{\"name\":\"_mp\",\"type\":\"uint256\"},
{\"name\":\"_str\",\"type\":\"uint256\"},{\"name\":\"_intelli\",\"type\":\"uint256\"},{\"name\":\"_san\",\"type\":\"uint256\"},
{\"name\":\"_luck\",\"type\":\"uint256\"},{\"name\":\"_charm\",\"type\":\"uint256\"},{\"name\":\"_mt\",\"type\":\"uint256\"},
{\"name\":\"_optionalAttrs\",\"type\":\"string\"}],\"name\":\"insertCharacter\",\"outputs\":
[{\"name\":\"\",\"type\":\"uint256\"}],\"payable\":false,\"stateMutability\":\"nonpayable\",\"type\":\"function\"},{\"constant\":true,\"inputs\":
[],\"name\":\"characterNo\",\"outputs\":[{\"name\":\"\",\"type\":\"uint256\"}],\"payable\":false,\"stateMutability\":\"view\",\"type\":\"function\"},
{\"constant\":false,\"inputs\":[{\"name\":\"_id\",\"type\":\"uint256\"},{\"name\":\"_hp\",\"type\":\"uint256\"},{\"name\":\"_mp\",\"type\":\"uint256\"},
{\"name\":\"_str\",\"type\":\"uint256\"},{\"name\":\"_intelli\",\"type\":\"uint256\"},{\"name\":\"_san\",\"type\":\"uint256\"},
{\"name\":\"_luck\",\"type\":\"uint256\"},{\"name\":\"_charm\",\"type\":\"uint256\"},
{\"name\":\"_optionalAttrs\",\"type\":\"string\"}],\"name\":\"setCharacterAttributes\",\"outputs\":
[],\"payable\":false,\"stateMutability\":\"nonpayable\",\"type\":\"function\"},{\"inputs\":
[],\"payable\":false,\"stateMutability\":\"nonpayable\",\"type\":\"constructor\"}]"}
```

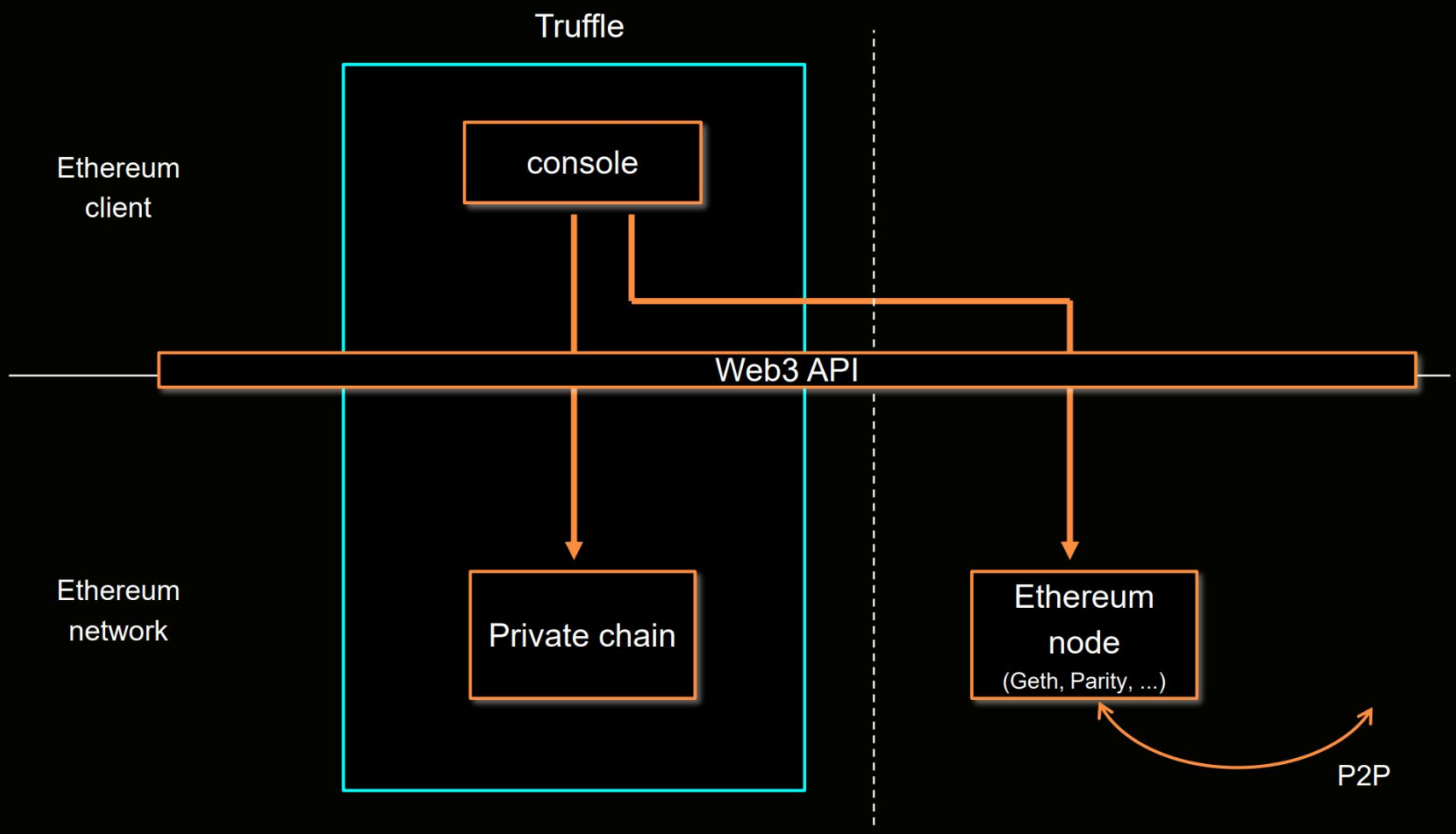

https://api.etherscan.io/api?module=contract&action=getabi&address=0x8cae244d8e274058d1c10c8ed731994a10b7e7d2

ABI for Last Trip

Geth

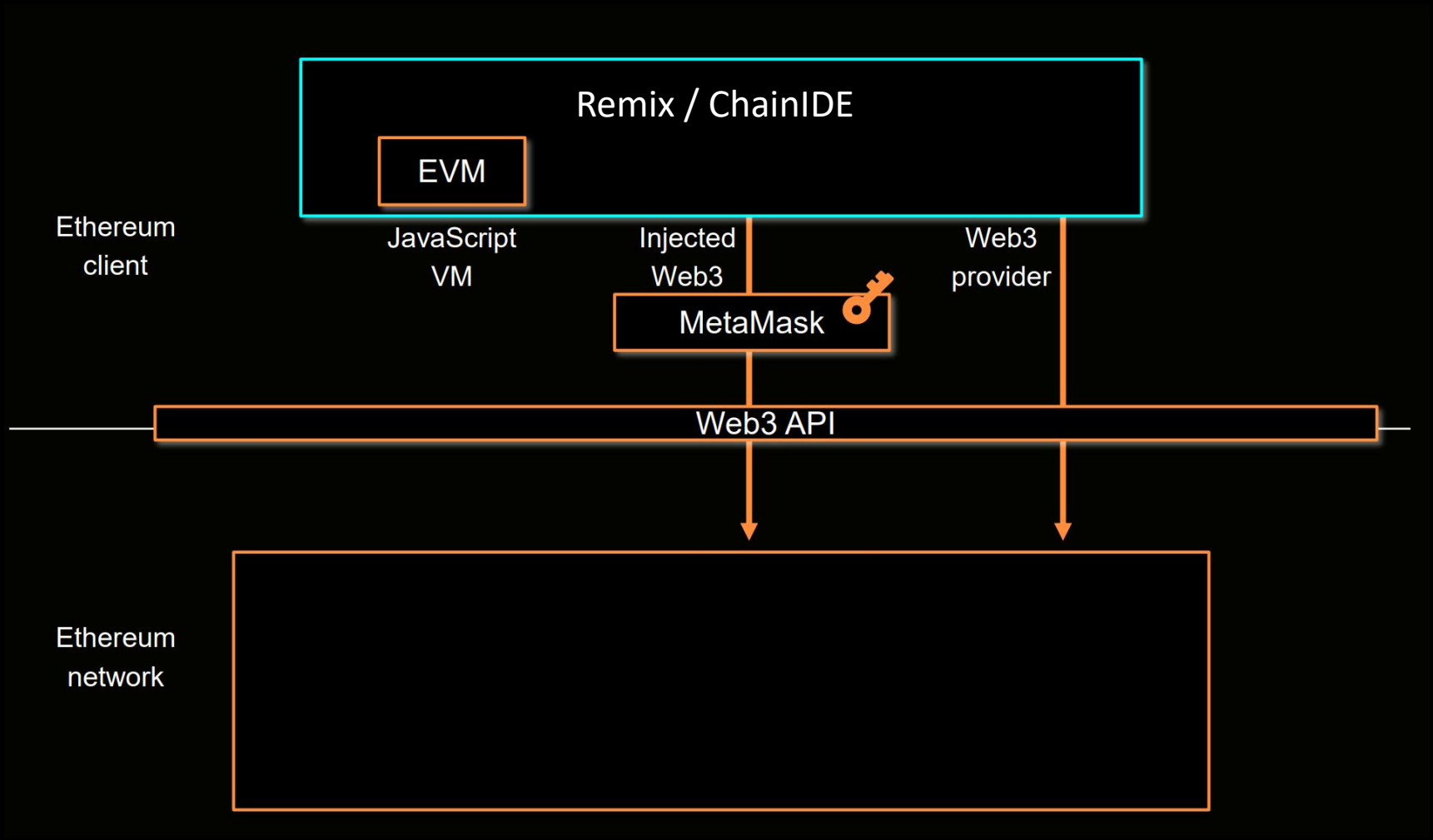

Geth
Ethereum client
console
Web3 API
node
Web3 I/F
Ethereum network
EVM
...
P2P
Ethereum node
(Geth, Parity, ...)
P2P

Truffle



https://www.trufflesuite.com/

Remix / ChainIDE



Ethereum Testnets

PoW Era: Ropsten and Rinkeby



Tech

Etherscan Set to 'Deprecate' Ethereum's Ropsten and Rinkeby Testnets

Developers should migrate to Goerli and Sepolia networks ahead of tomorrow's shutdown.

By Margaux Nijkerk Oct 5, 2022 at 3:17 a.m.

Now: Goerli and Sepolia

Front End: Web3 API

- web3.js: Ethereum JavaScript API
- A collection of libraries to allow you to interact with a local or remote Ethereum node using HTTP, IPC or WebSocket
- Can be used in node.js server or web browser with html&javascript
- Documentation
  - https://web3js.readthedocs.io/en/v1.8.0/

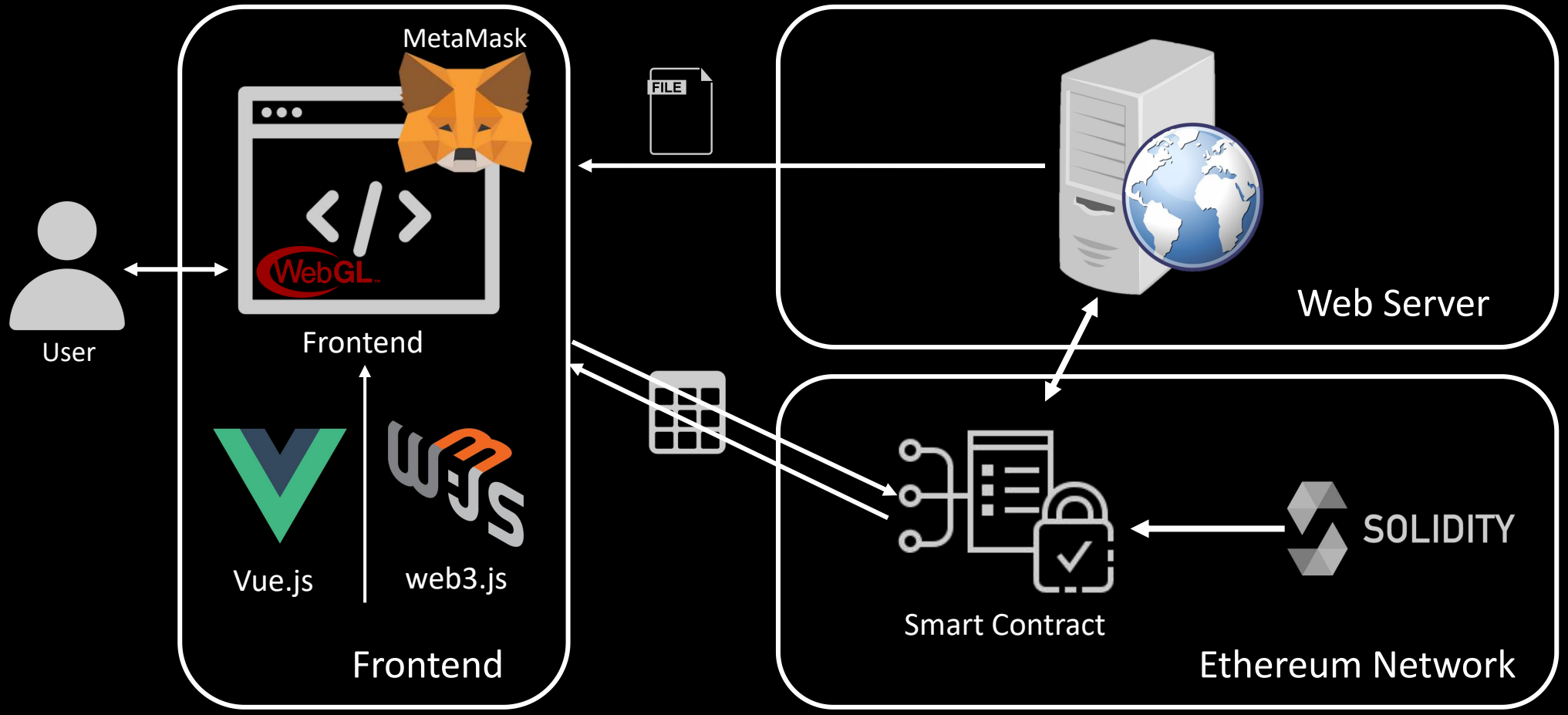
web3.js - Ethereum JavaScript API

web3.js is a collection of libraries that allow you to interact with a local or remote ethereum node using HTTP, IPC or WebSocket.

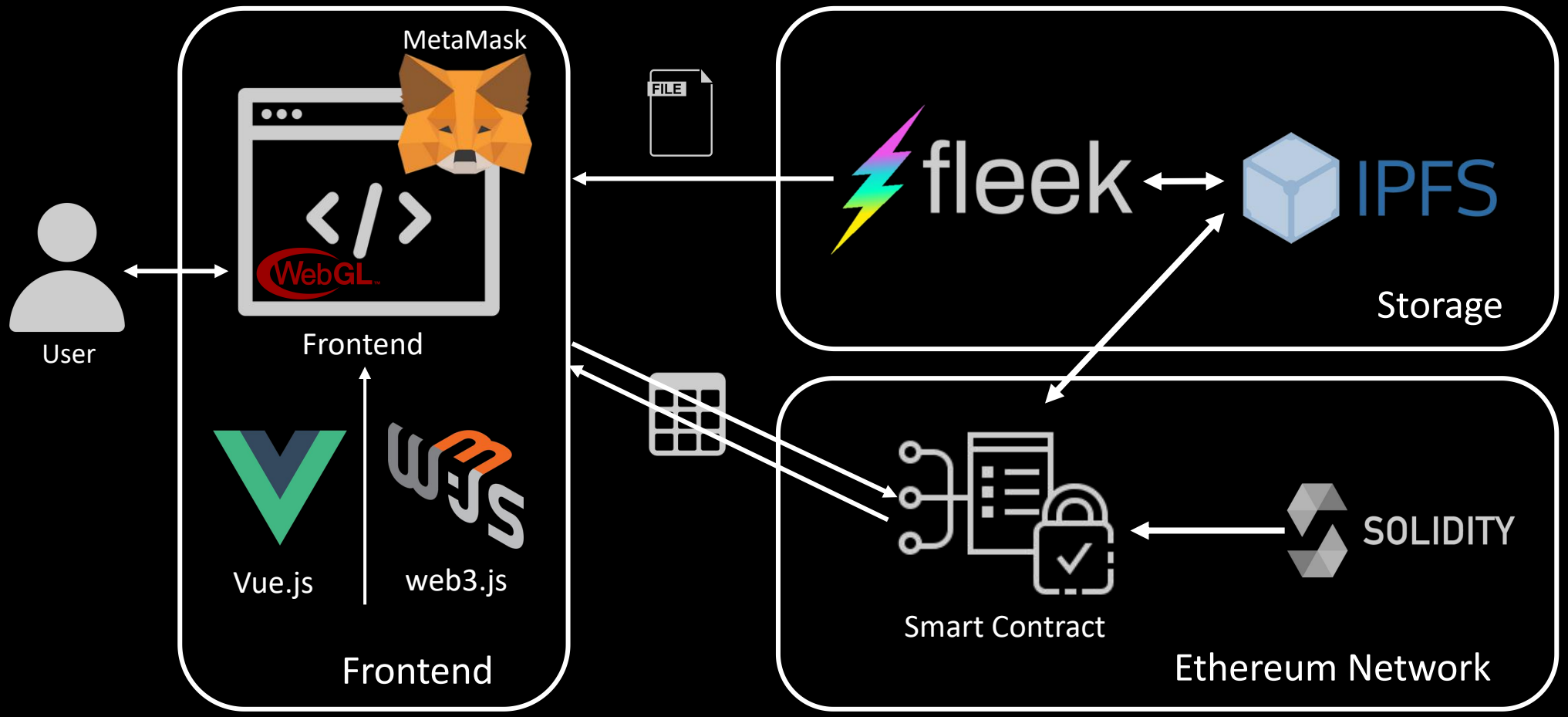
The following documentation will guide you through installing and running web3.js as well as providing an API reference documentation with examples.

Web3.js in 2 mins

Wait… who send you the html and js files?



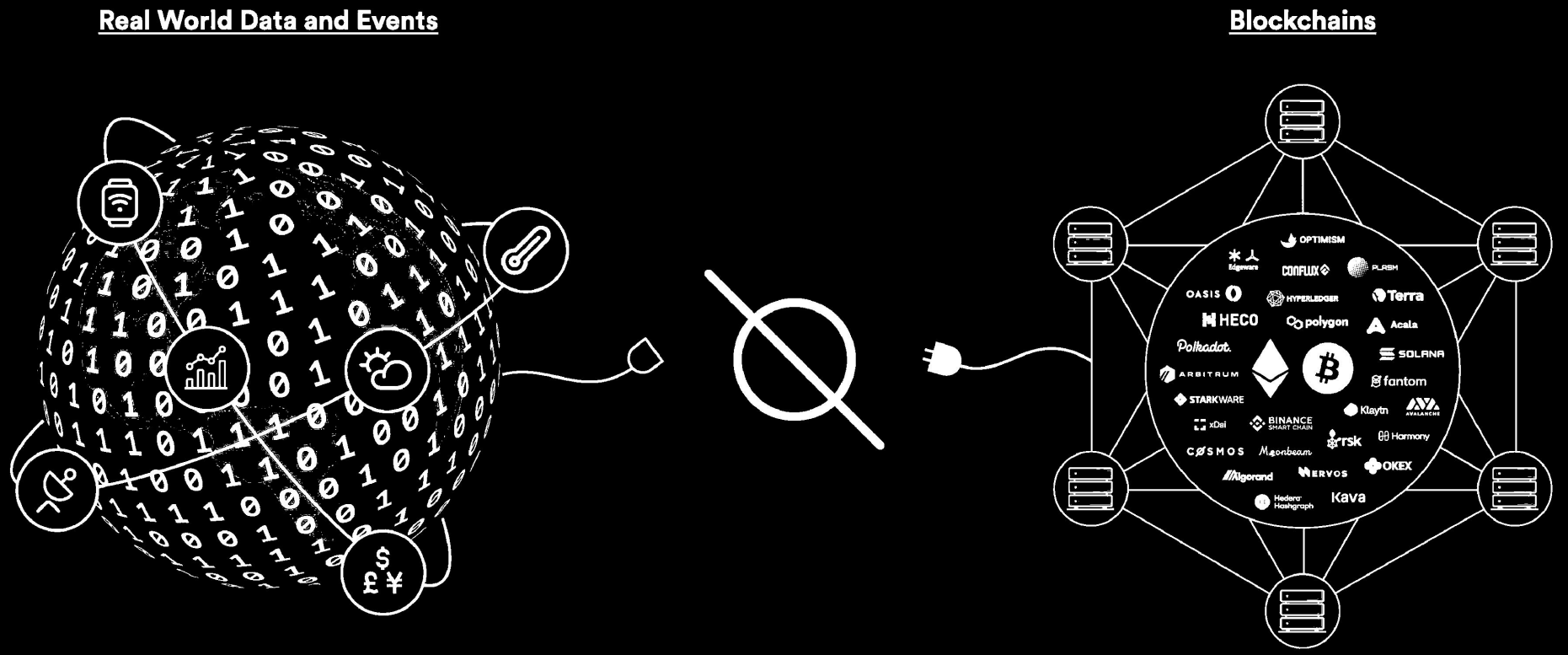
Fleek + IFPS: Better Decentralization





Fleek

The Oracle Problem

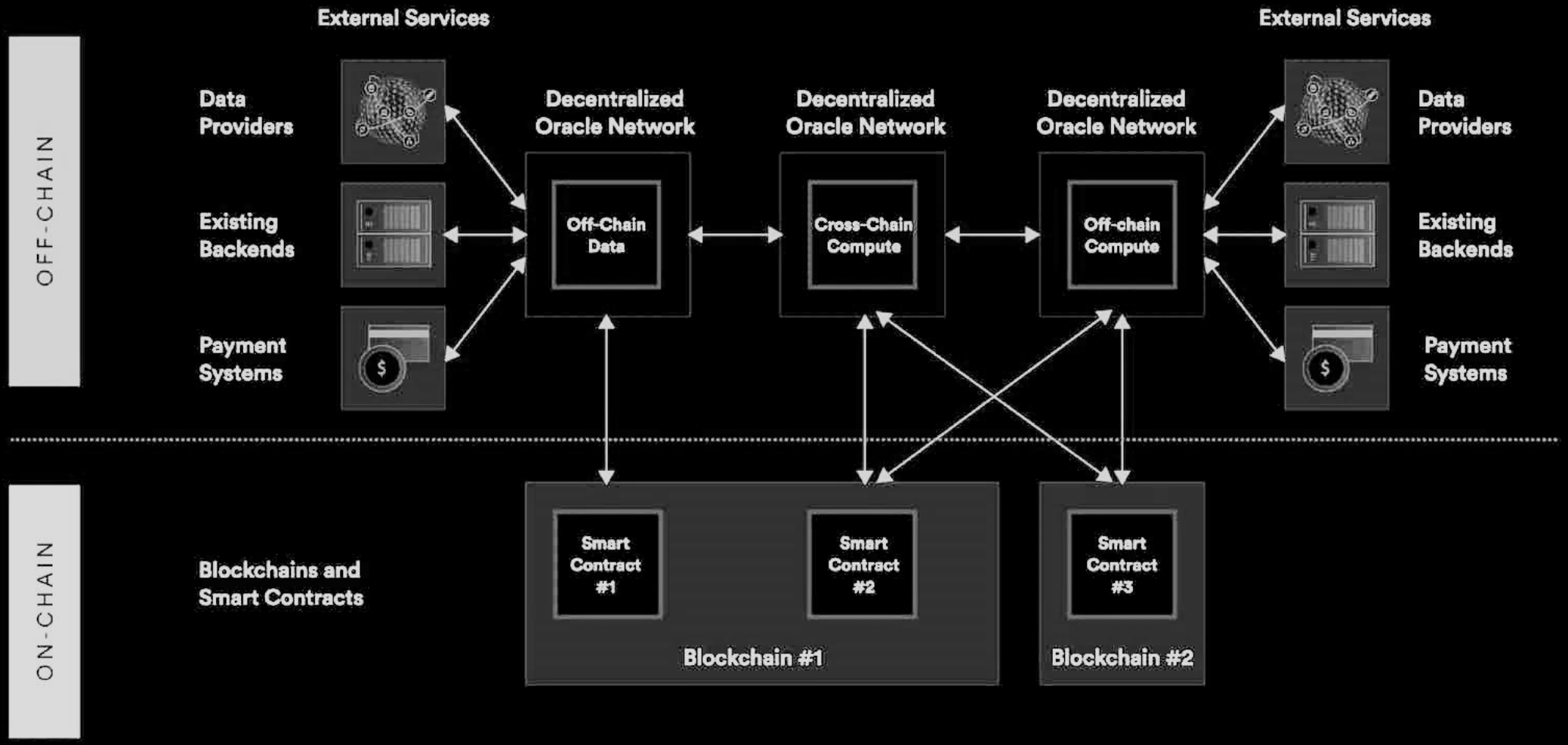


Smart contracts cannot inherently interact with data and systems existing outside their native blockchain environment

Blockchain Oracle

- Provide a way for the smart contract to access
  - real-world data sources
  - legacy systems
  - advanced computations
- Decentralized oracle networks (DONs)
  - hybrid smart contracts
  - Combine on-chain code and off-chain infrastructure
  - to support advanced decentralized applications (dApps)

ChainLink



https://chain.link

Now you all set

All you need is the COURAGE

to code



Extended Readings: Solidity Language

- Solidity By Example
  - https://solidity.readthedocs.io/en/v0.5.11/solidity-by-example.html#
- Solidity Documentation
  - https://docs.soliditylang.org/en/v0.8.17/

- Ethernaut
  - https://ethernaut.openzeppelin.com
- CryptoZombies
  - https://cryptozombies.io/

Extended Readings: Web IDE

- ChainIDE
  - https://eth.chainide.com/
- Remix
  - https://remix.ethereum.org/



Web3.js with Web Frontend